

uc3m

Universidad
Carlos III
de Madrid

Máster Universitario en Ingeniería Informática

Master Thesis

“Implementing Interrupts, Timers,
and Memory-Mapped I/O in
CREATOR”

Author

Luis Daniel Casais Mezquida

Advisor

Alejandro Calderón Mateos

Leganés, Madrid

February 2026



This work is licensed under Creative Commons
Attribution – Non Commercial – No Derivatives 4.0 International


```
git commit -m "minor updates"
```

— **Alejandro Calderón**

ABSTRACT

CREATOR is a didactic and generic assembly language simulator that simulates the behaviour of different Instruction Set Architecture (ISA). It is used in computer structure and computer architecture courses around the world as a tool where students can apply their theoretical knowledge in a friendly interface.

One of the most requested features by university professors is support for interrupts. Due to the current geopolitical and commercial climate, where chip design has become crucial for a country's economic independence, and open and modular standards such RISC-V thrive, understanding interrupts and timers—some of the bases of computer hardware—has become paramount. Furthermore, in a technological age in which high-performance computation through the use of specialized devices such as GPUs is becoming increasingly more important, understanding how these devices communicate and synchronize with the CPU is critical.

This thesis describes the design and implementation of these features in the CREATOR simulator, and the challenges and solutions found along the way.

Keywords: Computer science education • Digital simulation • Interrupts • I/O

ACKNOWLEDGEMENTS

It is tradition to thank family and friends first, so I'll start by thanking my parents. This time around they didn't "pay for the dammed thing"—I did, with my hard work, while also paying for hospitals and roads—but they did support me throughout. My friends also supported me, although they just listened to me while I screamed about JavaScript or some other crap, politely nodding. Still, they helped me relax by telling jokes or doing stupid stuff, and listened to my feelings and tales of my (somewhat) sad life. I would be very bored without all of them.

Although this thesis *is* a project, it was also part of a bigger project, which was also my job, at the ARCOS research group. There are some truly smart and wonderful people there, but I do have to name the ones I was actually *supposed* to be working and talking to. Alex, my tutor, is one of those trully wonderful persons that's good to its core. The guy just wants to learn about cool (computer) stuff, share the knowledge, and reap the benefits of seing people making even cooler stuff; but he also has a couple of left hands to deal with weird administrative and/or technical problems, finding solutions that nobody else would think of (for better or for worse). Félix, similarly to Alex, also offered great guidance on what the features were supposed to be doing, and where to go and search for the information about how to do it. Jorge, my *brother in arms*, was a great juxtaposition to my programming style and personality, and I probably learnt more from him than he did from me. I also have to thank Diego, CREATOR's creator (sic), who was always had the patience for answering my questions about the code, and Álvaro and Elisa, who worked on different parts of the project, but helped me when I needed to interact with them.

I also want to give a special thanks to *el GUL*. It's a wonderful set of (arguably weird) people willing to share their time, knowledge and excitement. My time has come to step down from the role of voluntary dictator, but I know I have left the position in good hands, and that I can come back 20 years from now and still find "Linux freaks" doing cool stuff together at this university.

I can't forget to thank my teachers, specially Carlos Galán (the great musician), and my fellow classmates, in particular Fran, Diego, and Lucas. They made this sixth year of my bachelor's degree more bearable.

Finally, I want to thank the free software community. Great people doing great things for the sake of doing them. Please God don't let AI slop ruin this.

TABLE OF CONTENTS

Chapter 1. Introduction	1
1.1. Motivation	1
1.2. Objectives	2
1.3. Document Structure	2
Chapter 2. State of the Art	3
2.1. Interrupts	3
2.1.1. Interrupt taxonomy	3
2.1.2. Interrupt detection and handling	4
2.1.3. Interrupt simulation	8
2.2. Timers	12
2.2.1. Timer implementations	13
2.2.2. Timer simulation	13
2.3. I/O devices	13
2.3.1. Memory-mapped I/O	13
2.3.2. I/O implementation and simulation	14
Chapter 3. Analysis	17
3.1. Project overview	17
3.2. Requirements	17
3.2.1. User Requirements	18
3.2.2. Software Requirements	21
3.2.3. Traceability	28
Chapter 4. Design, Implementation, and Deployment	31
4.1. Study of the Solution	31
4.1.1. Interrupts	32
4.1.2. Timers	34
4.1.3. Memory-mapped I/O	34
4.2. System Architecture	35
4.2.1. Interrupt Manager	35
4.2.2. Timer Manager	38
4.2.3. Device Manager	38
4.2.4. CAPI . INTERRUPTS	39
4.3. Implementation	40
4.4. Deployment	43

Chapter 5. Project Plan	45
5.1. Planning	45
5.1.1. Methodology	45
5.1.2. Time estimation	46
5.2. Budget	48
5.2.1. Direct costs	48
5.2.2. Indirect costs	49
5.2.3. Costs summary	50
5.2.4. Project offer proposal	50
5.3. Regulatory framework	50
5.3.1. Applicable legislation	50
5.3.2. Technical standards	51
5.3.3. Licenses	51
Chapter 6. Conclusions	53
6.1. Project conclusions	53
6.2. Personal conclusions	54
6.3. Additional contributions	54
6.4. Future work	55
Bibliography	57
Appendix A. Declaration of Use of Generative AI	

LIST OF FIGURES

Fig. 2.1	Vectored interrupts	5
Fig. 2.2	RV32's mstatus register	5
Fig. 2.3	RV32's mie register	5
Fig. 2.4	RV32's mip register	5
Fig. 2.5	RV32's mcause register	7
Fig. 2.6	RV32's mtvec register	7
Fig. 2.7	WepSIM's Elemental Processor	10
Fig. 2.8	WepSIM's Control Unit	10
Fig. 2.9	A simple memory-mapped register	14
Fig. 2.10	Partial address decoding of eight registers	14
Fig. 4.1	Core module architecture	35
Fig. 4.2	Interrupt Manager architecture	37
Fig. 4.3	Timer Manager	38
Fig. 4.4	Device Manager architecture	38
Fig. 4.5	Project file structure	41
Fig. 5.1	Spiral model life cycle	46
Fig. 5.2	Gantt chart for the project	47

LIST OF TABLES

Table 2.1	List of relevant mcause register values	7
Table 3.1	“User” requirement template	18
Table 3.2	Requirement “R-U-CA-01”	18
Table 3.3	Requirement “R-U-CA-02”	19
Table 3.4	Requirement “R-U-CA-03”	19
Table 3.5	Requirement “R-U-CA-04”	19
Table 3.6	Requirement “R-U-CA-05”	19
Table 3.7	Requirement “R-U-CA-06”	20
Table 3.8	Requirement “R-U-CA-07”	20
Table 3.9	Requirement “R-U-CA-08”	20
Table 3.10	Requirement “R-U-CA-09”	20
Table 3.11	Requirement “R-U-RE-01”	21
Table 3.12	Requirement “R-U-RE-02”	21
Table 3.13	Requirement “R-U-RE-03”	21
Table 3.14	“Software” requirement template	22
Table 3.15	Requirement “R-S-FN-01”	22
Table 3.16	Requirement “R-S-FN-02”	22
Table 3.17	Requirement “R-S-FN-03”	22
Table 3.18	Requirement “R-S-FN-04”	23
Table 3.19	Requirement “R-S-FN-05”	23
Table 3.20	Requirement “R-S-FN-06”	23
Table 3.21	Requirement “R-S-FN-07”	23
Table 3.22	Requirement “R-S-FN-08”	24
Table 3.23	Requirement “R-S-FN-09”	24
Table 3.24	Requirement “R-S-FN-10”	24
Table 3.25	Requirement “R-S-FN-11”	24
Table 3.26	Requirement “R-S-FN-12”	25
Table 3.27	Requirement “R-S-FN-13”	25

Table 3.28 Requirement “R-S-FN-14”	25
Table 3.29 Requirement “R-S-FN-15”	25
Table 3.30 Requirement “R-S-FN-16”	26
Table 3.31 Requirement “R-S-FN-17”	26
Table 3.32 Requirement “R-S-FN-18”	26
Table 3.33 Requirement “R-S-FN-19”	26
Table 3.34 Requirement “R-S-FN-20”	27
Table 3.35 Requirement “R-S-FN-21”	27
Table 3.36 Requirement “R-S-FN-22”	27
Table 3.37 Requirement “R-S-FN-23”	27
Table 3.38 Requirement “R-S-NF-01”	28
Table 3.39 Requirement “R-S-NF-02”	28
Table 3.40 Requirement “R-S-NF-03”	28
Table 3.41 Traceability Matrix, Functional vs Capability	29
Table 4.1 ConsoleDevice operations	39
Table 4.2 OSDriver operations	39
Table 5.1 Project information	48
Table 5.2 Personnel costs	49
Table 5.3 Equipment costs	49
Table 5.4 Indirect costs	50
Table 5.5 Costs summary	50
Table 5.6 Offer proposal	50

LIST OF ALGORITHMS

Algorithm 2.1	CREATOR 5's instruction execution cycle	9
Algorithm 2.2	WepSIM's interrupt hook	11
Algorithm 2.3	WepSIM's return from interrupt	11
Algorithm 2.4	WepSIM's instruction execution cycle	12
Algorithm 4.5	CREATOR 6's instruction execution cycle	31
Algorithm 4.6	CREATOR 6's interrupt handling subroutine	33
Algorithm 4.7	CREATOR 6's timer handling subroutine	34
Algorithm 4.8	CREATOR 6's device handling subroutine	35
Algorithm 4.9	Interrupt Manager resetting subroutine	36
Algorithm 4.10	Interrupt Manager syncing subroutine	37

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CAPI	CREATOR API
CLI	Command Line Interface
CPU	Central Processing Unit
FOSS	Free Open-Source Software
GPU	Graphics Processing Unit
GUI	Graphical User Interface
I/O	Input/Output
ISA	Instruction Set Architecture
IoT	Internet of Things
MMIO	Memory-Mapped Input/Output
PC	Program Counter
UI	User Interface

CHAPTER 1

INTRODUCTION

This first chapter briefly presents the project: why it came to be ([Section 1.2, Objectives](#)), the objectives it strives to fulfill ([Section 1.2, Objectives](#)), and a description of the overall structure ([Section 1.3, Document Structure](#)).

1.1. Motivation

CREATOR (*didaCtic and geneRic assEmbly progrAMming simulaTOR*) is an educational interactive platform for assembly programming. It is developed by the ARCOS research group at Universidad Carlos III de Madrid in order to aid in the teaching of the Computer Structure and Computer Architecture courses. Its focus is on providing a simple interactive experience for the students, while being able to support multiple architectures, or modify existing ones, without needing to create a new simulator for each language [1].

In 2010, RISC-V appears as an open, modular ISA [2]. This comes at a time when the commercial war between the USA and China has just started, affecting the exportation of microchips and, specially, GPUs. This forces countries and corporations to start investing in manufacturing their own chips, and an open standard such as the mentioned RISC-V can act as a base for designing and implementing their own. Furthermore, recent technological advances in cloud computing, IoT, and AI greatly benefit from hardware acceleration through the use of external devices, making communication with them critical. On the other hand, these advancements (and overall push) in computing has also put a great focus in parallelization, with one of the most basic mechanisms to moderate that being timers and interrupts. As the dependence on computing increases, the cybersecurity risks associated to them also increase, and many of these vulnerabilities and exploits occur at the assembly level. All of this creates a high demand for people with deep knowledge of assembly programming in general, and interrupts, timers and MMIO in particular. Currently, there

are not many didactic assembly simulators that support interrupts, timers, and/or MMIO¹, and virtually none that are also generic. Nevertheless, most university courses on computer architecture teach these key concepts, while lacking the tools to help the students visualize and learn.

This work is part of “*Integrated development environment for teaching and research on RISC-V processors*”, funded by the *Agencia Estatal de Investigación* (AEI) and by the European Union NextGenerationEU/PRTR [3].

1.2. Objectives

The main objective of this work is to expand the CREATOR simulator with support for interrupts, timers, and I/O devices, while keeping a generic approach that is flexible enough for defining multiple ISAs, and without defeating its educational purposes.

The secondary objectives, derived from the main objective, are as follows:

- **O1:** Simulate interrupts in a generic way, allowing the emulation of multiple ISAs.
- **O2:** Simulate a generic timer that can generate interrupts according to the ISA specification.
- **O3:** Simulate memory-mapped I/O devices that allow for different new features.
- **O4:** Integrate these new features in a modular simulator that is easy to maintain and expand.

1.3. Document Structure

This document contains the following chapters:

- [Chapter 1](#), *Introduction*, briefly presents the project, its motivations and objectives, and a description of the contents of the document.
- [Chapter 2](#), *State of the Art*, discusses the technological environment surrounding interrupts, timers, and I/O devices, including its hardware implementation and simulation.
- [Chapter 4](#), *Design, Implementation, and Deployment*, describes the design and implementation process, detailing the system’s architecture, and justifying the different decisions taken. It also outlines the necessary steps for its deployment.
- [Chapter 5](#), *Project Plan*, presents the concepts related to the followed planning, and breaks down all project costs. It also discusses the regulatory framework and socio-economic environment that applies to the project.
- [Chapter 6](#), *Conclusions*, highlights the contributions of the project, discusses the overall conclusions, and presents what future work could be done in order to improve the system.
- [Appendix A](#), *Declaration of Use of Generative AI*, describes the use of Generative AI in this work.

¹This will be detailed in [Chapter 2](#).

CHAPTER 2

STATE OF THE ART

This chapter presents the current stage of the technologies implemented in this project: interrupts (Section 2.1, *Interrupts*), timers (Section 2.2, *Timers*, and memory-mapped I/O (Section 2.3.1, *Memory-mapped I/O*, including how they are implemented in different ISAs and simulators.

2.1. Interrupts

Authors and vendors use different terms to refer to the same concept, a way of stopping the current execution of the program due to an exceptional situation. In this thesis, the term *interrupt* will be used.

2.1.1. Interrupt taxonomy

Patterson and Hennerly [4, chap. 4.10] use the term *exception* to refer to “[a]n unscheduled event that disrupts program execution; used to detect undefined instructions”, defining *interrupt* as “[a]n exception that comes from outside the processor”. Bulić [5, chap. 2.3.1], on the other hand, differentiates between *interrupts* and *traps*. He defines interrupts as “signals generated by hardware peripherals or devices to signal the processor of an event that requires attention”, and distinguishes between two types: *software* interrupts, which “function similarly to subroutine calls” and “are usually used for requesting operating system services”, and *external* interrupts, which are triggered by “[a]n external device [...] activating the interrupt request pin on the CPU”. He further divides the latter into *maskable*, which “can be enabled or disabled by setting a bit in the CPU’s control register or executing a special instruction”, and *non-maskable*, which “have a higher priority than maskable interrupts”. He also defines *traps* as “internally generated signals that indicate exceptional conditions or errors encountered during the execution of program executions” that “require special handling by the processor or operating system to resolve and recover from the error”.

Undergraduate courses on computer architecture and organization, such as the ones from the Northern Illinois University [6] and Universidad Carlos III de Madrid [7] classify *interrupts* as either *synchronous* or *asynchronous*. *Synchronous* interrupts are generated when an instruction fails as a result of an error, or by special system call instructions, and abort the execution of the current instruction; *asynchronous* interrupts (also called *interrupt requests*) are generated by either a device, or a timer, and wait for the instruction to finish. This distinction is mainly relevant in pipelined architectures, where one instruction is performed during several clock cycles, and an error during the execution of an instruction compromises the pipeline and requires immediate attention.

While the Z80 CPU only differentiates between *software maskable interrupts* and *nonmaskable interrupts* [8, p. 17], the RISC-V ISA uses the term *exception* as “an unusual condition occurring at run time associated with an instruction”, while an *interrupt* is “an external asynchronous event that may cause a RISC-V hart to experience an unexpected transfer of control”. It also defines a *trap* as “the transfer of control to a trap handler caused by either an exception or an interrupt”, and distinguishes between *contained traps*, where “[t]he trap is visible to, and handled by, software running inside the execution environment”; *requested traps*, where “[t]he trap is a synchronous exception that is an explicit call to the execution environment requesting an action on behalf of software inside the execution environment”, and “execution may or may not resume on the hart after the requested action is taken by the execution environment”; *invisible traps*, where “[t]he trap is handled transparently by the execution environment and execution resumes normally after the trap is handled”; and a *fatal trap*, which “represents a fatal failure and causes the execution environment to terminate execution” [9, chap. 1.6].

2.1.2. Interrupt detection and handling

CPUs that support interrupts have an interrupt request (*IRQ*) pin and an interrupt acknowledge (*INTA*) pin, sometimes mapped to a register. When an interrupt occurs, the *IRQ* pin is asserted. As the CPU should finish executing the current instruction before attending the interrupt, a device may have to wait for several clock cycles before it is attended, so the *INTA* pin is used to signal the device that the CPU has started servicing the interrupt request. The *IRQ* pin is typically checked before fetching a new instruction.

Before handling the interrupt, it is necessary to save the execution context, so that the program can be resumed when the interrupt handler has completed. The execution context varies depending on the architecture, and can include, stack pointer, status register, and any other registers that are visible to the CPU, but it always includes at least the program counter. This context can be stored on special registers, on the stack, or in a combination of both, depending on the architecture. The cause of the interrupt is also stored in architectures that support different types of interrupts.

Modern architectures, such as Intel’s x86 [10] and RISC-V [11], include *privileged modes* of execution. These modes enable multiple operating systems, hypervisors, and security models to be implemented in the same instruction set architecture, by restricting access to certain instructions and/or control registers to certain modes. The most basic implementation includes two modes: a *machine* mode and a *user* mode, to prevent user-

level applications to run with restricted access to system resources, although a third *supervisor* mode, designed for operating system kernels and with access to some privileged instructions and registers, is sometimes implemented. In these architectures, when a mode requires executing an action of higher privilege, such as accessing hardware devices, it generates an interrupt (also called *trap* in this specific case) that changes the execution mode. Interrupts generated by other methods, such as errors during the execution, also trigger this change.

Different types of interrupts require different procedures to handle them, therefore there must be a mechanism to execute the correct one for each type. There are two approaches to this problem: *polled interrupts* and *vectored interrupts*. In the former, the CPU branches to a specific handler address that typically evaluates the type of interrupt and branches to the handler for the specific type. The latter—as shown in Fig. 2.1—implements an *interrupt vector table*, a table containing *interrupt vectors*, addresses of interrupt handlers, typically managed by the operating system. The CPU jumps to each entry specified in the *interrupt register*. This register is typically divided into the base address of the interrupt vector table and the interrupt type, or interrupt exception code, allowing users to specify the address of the table. Once the handler has finished, usually signaled by some kind of special “return from interrupt” instruction, the CPU must restore the previous execution context and privilege mode. Most architectures include a special instruction to return from an interrupt with this functionality.

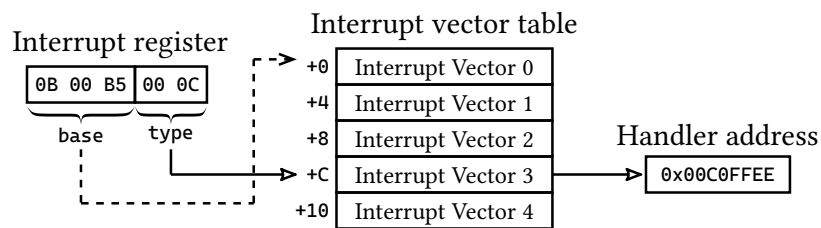


Fig. 2.1. Vectored interrupts

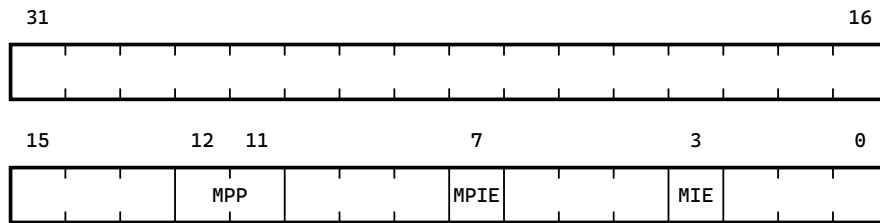
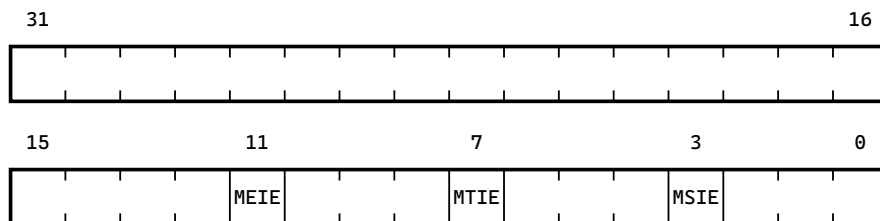
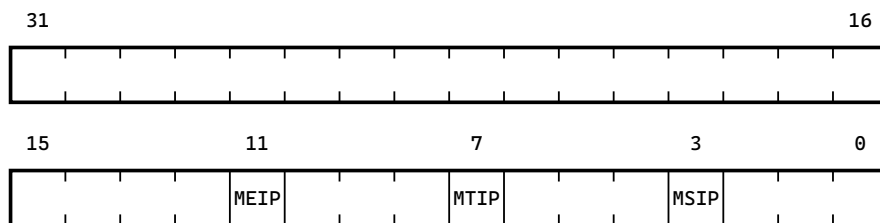
Interrupts on the RISC-V-32 ISA

As mentioned in Section 2.1.2, the RISC-V-32 ISA defines different privilege modes that determine the level of access and control a process has over the system’s resources levels, in particular, access to *Control Status Registers* (CSRs)², the privileged instruction set, and privileged ISA extensions. The ISA defines three levels: User/Application (U), Supervisor (S), and Machine (M), allowing implementations to provide one, two, or three of them³. M-level is the highest privilege level, while U-level is intended for conventional applications and S-level for operating systems [5, chap. 2.5.1], [11, chap. 1.2].

RISC-V handles exceptions and interrupts—as stated in Section 2.1.1—by generating traps. As a trap involves elevating the privilege level, e.g. requesting an OS system call from a user program, both M and S privilege levels include a set of CSRs for handling

²The set of instructions to control CSRs is provided in the *Zicsr* extension [9, chap. 7].

³There are only three supported combinations: M-level, M-level and U-level, or all three.

Fig. 2.2. RV32's `mstatus` registerFig. 2.3. RV32's `mie` registerFig. 2.4. RV32's `mip` register

them⁴. The `mstatus` (*Machine Status*) register (Fig. 2.2) keeps track of and controls the CPU's current operating state. In this register, there are several interrupt-related fields: field `MIE` (*Machine Interrupt Enable*) controls whether interrupts are globally enabled or disabled for that privilege mode, field `MPP` (*Machine Previous Privilege Mode*) holds the previous privilege mode, and, to allow nesting of interrupts, field `MPIE` stores the previous value of `MIE` when an interrupt occurs. The `mie` (*Machine Interrupt Enable*) register (Fig. 2.3) allows for a finer control of interrupts, allowing the programmer to set which types of interrupts are enabled: field `MSIE` (*Machine Software Interrupt Enable*), `MTIE` (*Machine Timer Interrupt Enable*), and `MEIE` (*Machine External Interrupt Enable*) control software, timer, and external interrupts, respectively. The `mip` (*Machine Interrupt Pending*) register (Fig. 2.4) indicates the interrupts that are currently pending. Similarly to register `mie`, it specifies the type of interrupt on specific fields: `MSIP` (*Machine Software Interrupt Pending*), `MTIP` (*Machine Timer Interrupt Pending*), and `MEIP` (*Machine External Interrupt Pending*). Each of these fields may be writable or may be read-only. Register `mcause` (*Machine Cause*, Fig. 2.5) provides information about the event that caused the trap. This register contains a field `I` specifying if the cause was an interrupt or an exception, while the rest of the bits are reserved for the exception code. RISC-V defines some of these exception codes (see Table 2.1), while others are left for the implementation to use. Register `mtvec` (*Machine Trap-Vector Base-Address*, Fig. 2.6) holds the trap vector configuration. Depending on the value of the `MODE` field, RISC-

⁴As they are virtually the same, only the M-level set will be described.

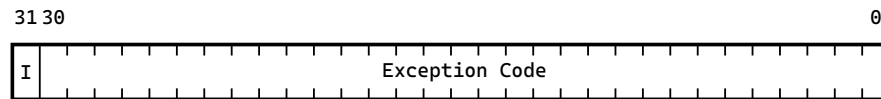


Fig. 2.5. RV32's mcause register

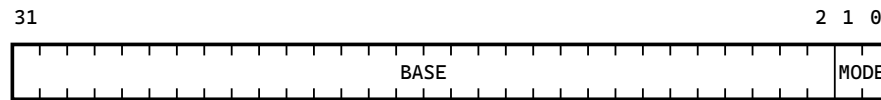


Fig. 2.6. RV32's mtvec register

TABLE 2.1

LIST OF RELEVANT mcause REGISTER VALUES

I	Exception Code	Description
0	8	Environment call from U-mode
1	3	Machine software interrupt
1	7	Machine timer interrupt
1	11	Machine external interrupt

V allows for polled interrupts (*direct mode*, with a value of 0) or vectored interrupts (*vectored mode*, with a value of 1). In direct mode, all traps cause the PC to be set to the address in the **BASE** field, while on vectored mode, traps set the PC to address $\text{BASE} + 4 \times \text{cause}$, *cause* being the exception code found in **mcause**. Finally, register **mepc** (*Machine Exception Program Counter*) holds the address of the instruction that generated the exception while it is handled [5, chap. 2.5.2], [9, chap. 3.1].

When a trap is taken to M-mode, the corresponding flag in **mip** is set, and the **MIE** field in **mstatus** and the corresponding flag in **mie** are checked to determine if that interrupt is enabled. If it's enabled, the **MIE** field is copied into the **MPIE** field of **mstatus** and **MIE** is cleared, disabling further interrupts. Then, the previous privilege mode is stored in the **MPP** field of **mstatus**, the cause of the exception is encoded into **mcause**, the current PC is stored into the **mepc** register, and the PC is set to the address specified by **mtvec**. When the interrupt handler finishes, it calls **mret**, which resets the privilege mode (reading the **MPP** field in **mstatus**), re-enables interrupts (by copying back field **MPIE** to **MIE** in **mstatus**), and sets the PC to the value of **mepc**. With regard to the **mip** register, the ISA states that if the field is writable, a pending interrupt can be cleared by clearing the field, but if the field is read-only, the implementation must provide some other mechanism for clearing the pending interrupt [5, chap. 2.5.4], [9, chap. 3.3].

Interrupts on the Z80 CPU

As discussed in Section 2.1.1, the Z80 CPU contains two types on interrupts, *maskable*, which can be enabled or disabled by the programmer, and *nonmaskable*, which can't be disabled. For this purpose, the CPU contains two interrupt input pins: **INT**, for maskable, and **NMI**, for nonmaskable. To enable interrupts, there is a flip-flop (**IFF**) that can be either set

or reset by the user using specialized instructions⁵. To account for nonmaskable interrupts, this flip-flop is split into two, IFF1 and IFF2. When a nonmaskable interrupt occurs (pin NMI is asserted), the status of IFF1 is stored in IFF2, and IFF1 is reset to prevent maskable interrupts from interrupting the handler. When a maskable interrupt occurs (pin INT is asserted) and is accepted, interrupts are disabled by clearing both IFF1 and IFF2 [8, pp. 17–18].

While nonmaskable interrupts are always handled by restarting the CPU at address 0x0066, maskable interrupts can be handled by one of the three possible modes, set by the programmer. In mode 0, the interrupting device places the next instruction on the data bus, and the CPU executes it. Mode 1 behaves similar to nonmaskable interrupts, by restarting at address 0x0038. Finally, mode 3 implements vectored interrupts, using a 16-bit pointer. The upper eight bits of the pointer are set by the contents of register I, while the lower eight bits are supplied by the interrupting device. As with regular routines, the address of the next instruction (current PC) is stored in the stack before branching to the handler routine, so the execution can be resumed [8, pp. 19–20, 59].

After the handler finishes, the interrupt pin must be reset⁶ before re-enabling interrupts, and returning from the routine. A RET instruction⁷ may be used to resume execution, but the Z80 ISA offers some alternatives with additional functionality for handling interrupts; the RETN (*RETurn from Nonmaskable interrupt*) instruction restores the original value of IFF1, while the RETI (*RETurn from Interrupt*) instruction is recognized by peripherals and automatically notifies them to reset the interrupt pin [8, pp. 18, 59–60].

As a concrete example, the Sinclair ZX Spectrum⁸'s ROM includes maskable and nonmaskable interrupt routines. While the nonmaskable routine forced a system reset, the maskable routine in charge of reading the keyboard, and used a RET instruction instead of RETI, due to the fact that these interrupts were generated by a timer and the INT pin was automatically reset [12, pp. 1–2].

2.1.3. Interrupt simulation

An *assembly simulator* is a piece of software that emulates a specific Instruction Set Architecture (ISA) CPU and allows the user to program it through its assembly language. Its purpose is usually educational, in order to aid the user in learning the language; or as a development tool, a quick way to iterate through different hardware designs without the cost of physically manufacturing components.

Most of the assembly simulators that exist are tailored to one specific ISA. In ISAs that have well-defined simple interrupt mechanisms, simulators such as YAZE-AG [13] just implement interrupts as described. On the other hand, ISAs that give the manufacturer flexibility to implement its features, like the RISC-V ISA, varie more. While SAILATOR

⁵EI, *Enable Interrupt*, and DI, *Disable Interrupt*.

⁶In the case that the interrupt was generated by a peripheral device, the device must be notified through the use of an OUT instruction, as it will be shown in Section 2.3.2.

⁷The RET instruction pops the top of the stack and stores it into the PC.

⁸Released in 1982 in the UK, it is one of the most popular computers that featured the Z80 family of CPUs.

[14], based on SAIL’s RISC-V ISA definition⁹ [16] implements the registers described in Section 2.1.2 and bundles its own kernel to handle traps, Spike [17], one the reference RISC-V simulators, supports Core Local Interrupt (CLINT) controller [18], and implements the relevant CSRs as memory-mapped registers. Other simulators, like Venus [19], support the Zicsr extension and include those registers, but don’t support interrupts, while simulators like [20] don’t even support that extension.

There are also simulators that support multiple ISAs, called *generic simulators*, although there are many approaches. Asm Editor [21] is a multi-ISA simulator that uses other simulators as a base; specifically for RISC-V, it uses RARS [22]. It doesn’t implement M-mode, only U-mode, and, according to its manual, “The implementation of user-mode exception handlers in RARS is not complete and does not fully conform to the RISC-V privileged specification, but it is based upon that [...]”. The simulator checks the `utvec`¹⁰ CSR is set, and if not, the trap is handled by the simulator itself, either by printing an error or performing some action in the case of OS calls (such as `ecall`). Another approach is to build a generic simulator that either allows the definition of the ISA, as is the case of CREATOR [1], or simulates a generic processor that allows for the definition of the microcode, such as WepSIM [23].

Interrupt simulation in CREATOR

CREATOR (*didaCtic and geneRic assEmbly progrAmming simulaTOR*) is an educational integrated development environment for assembly programming, developed by the ARCOS group at UC3M. CREATOR version 3.3 included some initial support for polled interrupts. It allows putting special tags in registers in the ISA definition: an *interrupt cause* (CAUSE) register, which signals that an interrupt occurred when its value is different from `0x0`, and an *exception program counter* (EPC), which stores the address of the interrupted instruction (*program counter*, PC). As shown in Algorithm 2.1, in CREATOR 5¹¹’s instruction execution cycle, interrupts are checked before fetching the instruction, therefore if an instruction generates an interrupt, the next instruction is the one to get interrupted. These registers were only implemented in the MIPS-32 architecture provided with the simulator.

Interrupt simulation in WepSIM

WepSIM (the *Web Elemental Processor SIMulator*) is an educational CPU simulator that emulates what the authors call the “Elemental Processor” (Fig. 2.7), a simple 32-bit processor with a (micro)programmable control unit (Fig. 2.8). This allows the user to define not only what signals on each instruction sets, but also how the instruction fetch is performed. For the purpose of asynchronous interrupt simulation, the control unit includes, connected to the Control Bus, an interrupt request pin (INT) and an interrupt acknowledge pin (INTA). The processor also includes a global interrupt enable *flag* (I), and an execution mode *flag* (U), although they are currently unused in the simulator.

⁹SAIL [15] is a definition language for ISAs, that also can generate a simulator from the definition.

¹⁰The equivalent of `mtvec` for U-mode.

¹¹As it will be mentioned in Section 4.3, the process of implementing these new features in the simulator, among other factors, involved an important rewrite of all its systems. The term “CREATOR 5” will be used to refer to the state of the project before these changes (version 5.0), and “CREATOR 6” will be used to refer to the state after the changes (version 6.0).

Algorithm 2.1: CREATOR 5's instruction execution cycle

```

1: if CAUSE  $\neq$  0 then                                 $\triangleright$  Interrupt detection
2:   EPC  $\leftarrow$  PC
3:   PC  $\leftarrow$  0
4:   CAUSE  $\leftarrow$  0
5: end if
6: Fetch instruction
7: Decode instruction
8: Increment PC
9: Execute instruction

```

As mentioned, the user can customize how the Elemental Processor functions, but WepSIM's handbook [25, chap. 2.5.], as well as some provided examples¹², offer some guidance with regard to *how* interrupts should be implemented. WepSIM distinguishes between three types of interrupts: *exceptions*, errors during the execution of an instruction; *system calls*, a special instruction that executes a handler in privileged mode; and *I/O interrupts*, caused by an I/O device. The convention when an interrupt happens (the *interrupt hook*)—as shown in Algorithm 2.2—is to store the program counter (PC) and status register (SR) in the stack (marked by the stack pointer register, SP), and jump to the interrupt handler. WepSIM implements vectored interrupts where, similarly to the mechanism described in

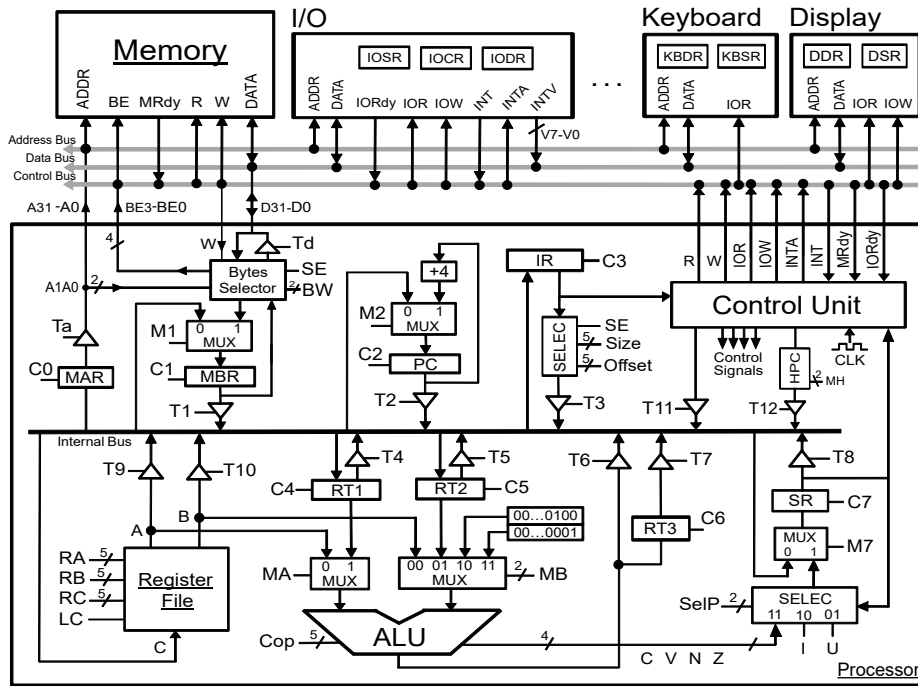


Fig. 2.7. WepSIM's Elemental Processor [24]

¹²Specifically, examples 9–13 in the “MIPS” example set.

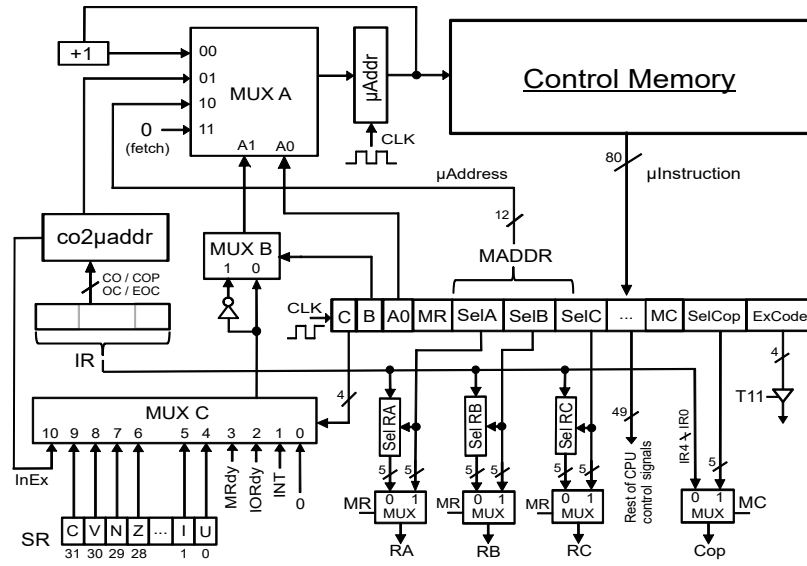


Fig. 2.8. WepSIM's Control Unit [24]

Fig. 2.1, register RT1 holds the interrupt type¹³, and the interrupt vector table's base address is always 0×0 . This value is then multiplied by 4 to obtain the handler address from memory, and then it stores it in PC. The vector table must be explicitly set by the user in the “kernel data” memory segment, through the use of the `.kdata` assembly directive. At the end of the interrupt handler, the *return from interrupt* (`reti`) instruction must be executed in order to restore the PC and SR, as shown in Algorithm 2.3.

Both exceptions and system calls are implemented as synchronous interrupts; the INT signal is not activated, and the instruction is responsible for storing the correct interrupt

Algorithm 2.2: WepSIM's interrupt hook

- 1: $\text{MEM}[\text{SP}] \leftarrow \text{PC}$
 - 2: Increment SP
 - 3: $\text{MEM}[\text{SP}] \leftarrow \text{SR}$
 - 4: Increment SP
 - 5: $\text{PC} \leftarrow \text{MEM}[\text{RT1} \times 4]$
-

Algorithm 2.3: WepSIM's return from interrupt

- 1: $\text{SR} \leftarrow \text{MEM}[\text{SP}]$
 - 2: Increment SP
 - 3: $\text{PC} \leftarrow \text{MEM}[\text{SP}]$
 - 4: Increment SP
-

¹³By convention, values 0×00 to 0×08 are reserved for exceptions and system calls, while values 0×09 to $0 \times \text{FF}$ are reserved for I/O devices.

Algorithm 2.4: WepSIM's instruction execution cycle

```

1: if  $INT \neq 0$  then                                ▷ Interrupt detection
2:    $INTA \leftarrow 1$ 
3:    $RT1 \leftarrow INTV$ 
    $INT \leftarrow 0$                                 ▷ Caused by I/O device
4:   Interrupt hook (Algorithm 2.2)
   end if
5: Fetch instruction
6: Increment PC
7: Decode instruction
8: Execute instruction

```

type in $RT1$ and performing the interrupt hook. On I/O interrupts, in the instruction execution cycle (shown in [Algorithm 2.4](#)), before performing the fetch, the Control Unit checks the INT signal. If it's active, the Control Unit acknowledges it by activating $INTA$, stores the interrupt type ($INTV$) provided by the I/O device in $RT1$, and performs the interrupt hook ([Algorithm 2.2](#)). Upon receiving the $INTA$ signal, the I/O device clears the INT signal.

2.2. Timers

Most computer applications require some a way to measure time intervals, either for scheduling tasks, polling for user inputs, or correctly implementing communication protocols. Most of these can be accomplished through software, by performing what's called “busy waiting”—using a dedicated thread to keep track of the elapsed time (for example, by incrementing a counter until a value is reached). Unfortunately, these *software timers* are not only inefficient—as you have one less thread available—but also inaccurate, as both the number of cycles an instruction will take and the clock frequency that determines the duration of those cycles are not consistent.

Another approach is to use a *hardware timer*, a dedicated electrical circuit that can keep track of time durations and emit signals when deadlines are met. All modern CPUs include a main system clock generated by a hardware oscillator, which is typically derived through the use of frequency multipliers to produce hardware clocks for other elements. Modern computers also include a programmable interval timer (PIT) that, as its name suggests, allows to programmatically generate timer interrupt signals. Another option that some computers, particularly embedded systems, employ are digital counters: registers whose value increments each *tick*, each time they are feed an input signal from a hardware clock. They typically can be configured to generate an interrupt once they reach a specific value, or when they overflow, and there can be multiple instances with different frequencies [[26](#), [chap. 9.5](#)], [[27](#), [chap. 22.1](#)].

2.2.1. Timer implementations

Most modern timer implementations involve, as we mentioned before, a PIT that is typically configured on startup through the use of CSRs, which generate an interrupt that, through the use of vectored interrupts, effectively acts as a “callback”—a function that the user sets and is *called back* when the deadline is met. This is generally managed in PCs by the OS, where the user can set these timers through system calls; and in the case of embedded systems without OS, the specific pins are set by the programmer.

One of the simplest implementations is the ZX Spectrum, which uses an internal clock to generate a maskable interrupt with a frequency of 50Hz¹⁴, which is used for the ‘keyboard’ routine [12, pp. 2–6]

On the other hand, the RISC-V ISA (both in its 32-bit and 64-bit variants) defines two 64-bit “machine timer” memory-mapped CSRs: `mtime` and `mtimecmp`. As stated in the specification, “`mtime` must increment at constant frequency, and the platform must provide a mechanism for determining the period of an `mtime` tick. The `mtime` register will wrap around if the count overflows”. To signal the deadline has been met, “[a] machine timer interrupt becomes pending whenever `mtime` contains a value greater than or equal to `mtimecmp`”, where “[t]he interrupt remains posted until `mtimecmp` becomes greater than `mtime`”. As stated in Section 2.1.2, these interrupts can be enabled or disabled by setting the MTIE flag in the `mie` register [11, chap. 3.2.1].

2.2.2. Timer simulation

Of the simulators mentioned in Section 2.1.3, only SAILATOR [14] and Spike [17] implement RISC-V’s timers. On Z80 CPU simulators, as timers are not described in the ISA, don’t usually have any timer support, but most ZX Spectrum simulators do generate a maskable interrupt on a 50Hz frequency, as the keyboard routine requires it. WepSIM includes an timer device—named “I/O”—that can be configured to generate interrupts every n clock cycles, as shown in Example 21.

2.3. I/O devices

The way a computer interacts with its environment is through the use of *Input/Output* devices, also called *peripherals*, and the interface where that interaction takes place is memory. This allows for one single CPU to interact with multiple different hardware devices, without having to add specific hardware to the CPU, as the software is in charge of handling that interaction. There are two main approaches for addressing this memory: using a portion of the system’s memory address space (*memory-mapped*), and exchanging data through simple load and store memory operations; and using a separate memory address space, also called “ports”, which require special input/output instructions [28].

2.3.1. Memory-mapped I/O

The latter is the most commonly used in modern computer systems, as it simplifies the interaction for software developers. All of these devices incorporate an amount of memory

¹⁴For the UK models. The NTSC/PAL models used a 60Hz frequency due to the different display refresh rate.

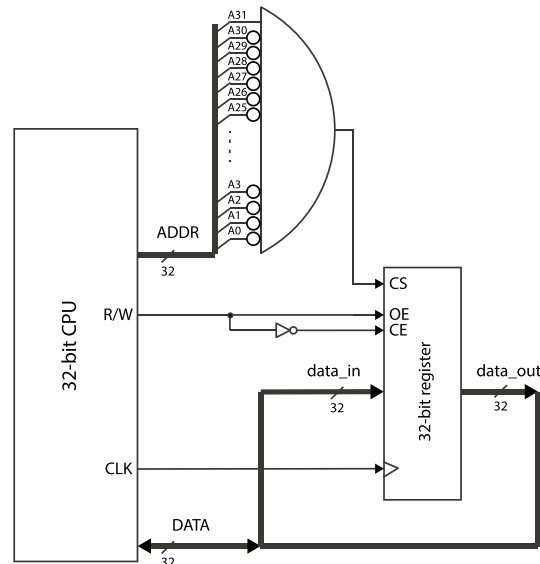


Fig. 2.9. A simple memory-mapped register [5, fig. 1.4]

words for data transfer and control, and each location is called a *register*, pictured in Fig. 2.9. These registers typically include, besides the standard data input/output and clock signals, three main inputs: *chip-enable* (CE, which controls whether data from the input data bus is stored to the register), *output-enable* (OE, which controls whether the data from the register is put in the output data bus), and *chip-select* (CS, which enables the register). When a CPU wants to read or write to memory, it activates its R/W signal and takes control of the DATA bus. This signal is directly connected—as shown in Fig. 2.9—to the OE input of all registers, as well as the inverted signal to the CE input, so that when a write operation is requested (R/W signal set high), the data is stored to the register, and if it's low, the value from the register is sent to the bus. In order to select which register is selected, a decoder is connected to the CS signal, so that the register is only enabled if it's the one that is addressed [5, chap. 1.1–1.2].

For decoding these addresses, there are two main methods: *full address decoding*, where the circuit tries to match the whole address, as is the case in Fig. 2.9; and *partial address decoding*, where the circuit only matches a subset of bits—typically the most significant bits—to determine if the address belongs to the device. To map registers from the same device consecutively in partial address decoding—useful for creating abstractions in software—the least significant bits are also matched with simple AND gates, as shown in Fig. 2.10. Partial address matching is the preferred way of implementing devices in hardware, as it requires less logic gates to implement [5, chap. 1.3–1.5].

2.3.2. I/O implementation and simulation

Different ISAs implement devices in different ways. Understanding these differences is essential for building robust simulators that can mimic the complex characteristics of real-world hardware.

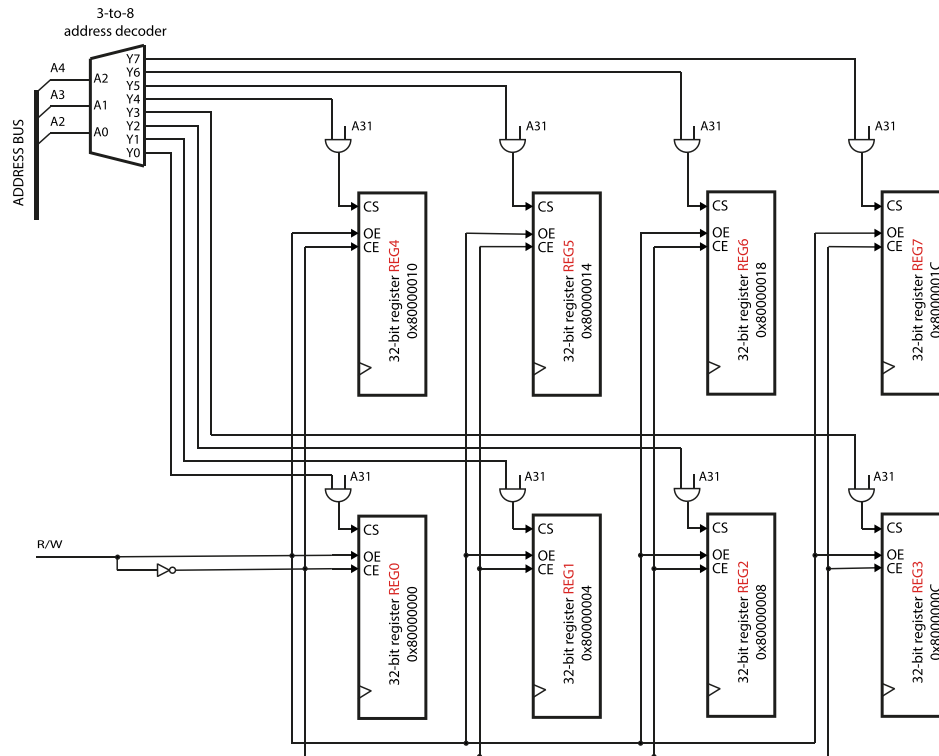


Fig. 2.10. Partial address decoding of eight registers [5, fig. 1.9]

Devices in RISC-V

RISC-V implements *Memory-Mapped I/O* (MMIO). In particular, the specification states that the *execution environment interface* (EEI) “will define what I/O operations are possible, and in particular, which memory addresses when accessed by load and store instructions will be treated and ordered as device input and device output operations respectively rather than memory reads and writes” [9, chap. 2.7].

For example, the FE310-G002 RISC-V based *System-on-Chip* (SoC)—based on the RV32IMAC ISA—uses memory-mapped registers for its GPIO interface, with four registers (input/output value and input/output enable) that control its 32 pins [5, chap. 1.7]. In terms of simulators, only Spike and Sail implement some platform-defined MMIO [16], [17]. SAILATOR implements the basic keyboard and display operations through the use of system calls, which directly interact with the web application [14].

Z80 peripherals

The Z80 CPU interfaced with devices through up to 256 ports, through the **IN** and **OUT** instructions [8]. The Sinclair ZX Spectrum used this for, among other devices, its screen and keyboard. For the screen, there were two modes of operation. The simplest way was intended for text output, as the system could write the values of specific characters directly to the screen. On the other hand, the system reserved one byte per pixel of main memory, and the ULA (Uncommitted Logic Array) was the chip in charge of rendering the screen by reading the memory segment, which was triggered by writing to ULA port. For the keyboard, an input port codifies the keys that are currently pressed, and a subroutine

triggered by an interrupt on a timer—as mentioned in [Section 2.2.1](#)—reads those values [12], [29], [30].

Devices in WepSIM

WepSIM’s Control Unit ([Fig. 2.8](#)) includes two pins connected to the control bus—IOR and IOW—in order read or write from/to a device register, similar to the OE and CE pins mentioned in [Section 2.3.1](#). As shown in [Fig. 2.7](#), the keyboard and display devices only have a data register and a status register, but the rest of the devices include an extra control register. The registers are addressed by reading the 2-byte port address from the address bus, and they use partial addressing, where the first byte selects the device and the last byte, the register. WepSIM devices are implemented on their own separate memory address space and use specific *in/out* instructions¹⁵. Nevertheless, the devices have *Direct Memory Access*, in order to read data from main memory, although there are no specific signals to control that feature [[25, chap. 2.5–2.6](#)].

As described in [Section 2.3.2](#), the timer device—with a device ID of 0x11—generates an interrupt if its IOCR—port 0x1104—is set to 0x0 every n cycles, n being the value of its IODR—port 0x1108. The keyboard device, when a key is pressed, stores in its status register (KBSR) the value 0x1, and the ASCII value of the key in its data register (KBDR), and the status register is automatically reset after the data register is read. The display device, on the other hand, automatically displays the ASCII character stored in its data register (DDR) and signals through its status register (DSR) that the operation has finished. Other devices, such as the LED matrix, read data from the main memory address stored in its data register [[25, chap. 2.5–2.6](#)].

¹⁵As the microinstructions are programmable, a user *could* implement devices as memory-mapped by modifying the read/write memory instructions.

CHAPTER 3

ANALYSIS

This chapter describes the proposed solution by briefly recapping the project ([Section 3.1, Project overview](#)), and specifying the system's requirements ([Section 3.2, Requirements](#)).

3.1. Project overview

The main objective of this project is to create an application that is capable of simulating interrupts, timers, and MMIO according to an user-defined ISA, for educational purposes. This will help the user learn these basic concepts by incentivizing experimentation and, in turn, giving the user a deeper understanding. As stated in [Chapter 2](#), current simulators either don't offer these features, or offer a small, non-configurable subset of them. The aim is to provide the user with an interactive approach to understanding these concepts.

3.2. Requirements

This section provides a detailed description of the system's requirements. For the requirement specification task, the IEEE recommended practices [31] were followed. According to these practices, a good specification must address the software functionality, performance issues, external interfaces, other non-functional features and design or implementation constraints.

Moreover, the requirement specification must be:

- **Complete:** The document reflects all significant software requirements.
- **Consistent:** Requirements must not generate conflicts with each other.
- **Correct:** Every requirement is one that the software shall meet according to the user needs.
- **Modifiable:** The structure of the specification allows changes to the requirements in a simple, complete and consistent way.
- **Ranked based on importance:** Every requirement must indicate its importance.

- **Traceable:** The origin of every requirement is clear, and it can be easily referenced in further stages.
- **Unambiguous:** Every requirement has a single interpretation.

Starting from the user requirements (Section 3.2.1), which constitute an informal reference to the product the client expects, the software requirements (Section 3.2.2) were derived. These requirements guided the design process with specific information on the functionality of the system, as well as any and other related characteristics.

3.2.1. User Requirements

This section provides a detailed description of the user's requirements for the project. These requirements indicate the main functionality and restrictions the developed system must fulfill. The user requirements are divided into two distinct types:

- **Capabilities:** Describe the expected system's functionality.
- **Restrictions:** Impose constraints or conditions that the system must fulfill.

Each user requirement is uniquely identified by an ID, which follows the format *R-U-YY-XX*, where *YY* identifies the type of the requirement—either a capacity (*CA*) or a restriction (*RE*)—and *XX* identifies the sequential number of the requirement within that type, starting at 01. Table 3.1 provides the template used for the specification of the requirements, including the description of each attribute.

TABLE 3.1
“USER” REQUIREMENT TEMPLATE

Field	Description
Description	Detailed description of the requirement
Necessity	Priority of the requirement of the user (<i>High, Medium, Low</i>)
Priority	Priority of the requirement for the developer (<i>High, Medium, Low</i>)

TABLE 3.2
REQUIREMENT “R-U-CA-01”

Field	Description
Description	The system shall be able to emulate any ISA's interrupt implementation.
Necessity	High
Priority	High

TABLE 3.3
REQUIREMENT “R-U-CA-02”

Field	Description
Description	The user shall be able to define the ISA’s interrupt implementation.
Necessity	High
Priority	Medium

TABLE 3.4
REQUIREMENT “R-U-CA-03”

Field	Description
Description	The system shall be able to emulate any ISA’s timer implementation.
Necessity	Medium
Priority	Medium

TABLE 3.5
REQUIREMENT “R-U-CA-04”

Field	Description
Description	The user shall be able to define the ISA’s timer implementation.
Necessity	Medium
Priority	Low

TABLE 3.6
REQUIREMENT “R-U-CA-05”

Field	Description
Description	The system shall support two privilege levels of execution: user and kernel.
Necessity	Medium
Priority	Low

TABLE 3.7
REQUIREMENT “R-U-CA-06”

Field	Description
Description	The system shall support system calls without requiring the usage of interrupts.
Necessity	Medium
Priority	Medium

TABLE 3.8
REQUIREMENT “R-U-CA-07”

Field	Description
Description	The system shall support memory-mapped devices.
Necessity	High
Priority	Medium

TABLE 3.9
REQUIREMENT “R-U-CA-08”

Field	Description
Description	The system’s memory-mapped devices shall provide the same capabilities as the system calls.
Necessity	Medium
Priority	Medium

TABLE 3.10
REQUIREMENT “R-U-CA-09”

Field	Description
Description	The user shall be able to define the ISA’s device addresses.
Necessity	High
Priority	Medium

TABLE 3.11
REQUIREMENT “R-U-RE-01”

Field	Description
Description	The system’s source code shall follow modern standards.
Necessity	High
Priority	Medium

TABLE 3.12
REQUIREMENT “R-U-RE-02”

Field	Description
Description	The system’s source code shall be maintainable.
Necessity	High
Priority	High

TABLE 3.13
REQUIREMENT “R-U-RE-03”

Field	Description
Description	The system shall be licensed under the LGPL-2.1 license.
Necessity	High
Priority	Low

3.2.2. Software Requirements

This section provides a detailed description of the system’s software requirements for the project. These requirements are derived from the user requirements, defined in [Section 3.2.1](#), and comprise the software specifications for the system. The software requirements are divided into two distinct types:

- **Functional requirements:** Specify the software’s functionalities and characteristics.
- **Non-functional requirements:** Specify other non-functional characteristics of the software.

Each software requirement is uniquely identified by an ID, which follows the format *R-S-YY-XX*, where *YY* identifies the type of the requirement—either functional (*FC*) or non-functional (*NF*)—and *XX* identifies the sequential number of the requirement within that type, starting at *01*. [Table 3.14](#) provides the template used for the specification of the requirements, including the description of each attribute.

TABLE 3.14
“SOFTWARE” REQUIREMENT TEMPLATE

Field	Description
Description	Detailed description of the requirement
Necessity	Priority of the requirement of the user (<i>High, Medium, Low</i>)
Priority	Priority of the requirement for the developer (<i>High, Medium, Low</i>)

TABLE 3.15
REQUIREMENT “R-S-FN-01”

Field	Description
Description	The system shall be able to emulate any ISA’s interrupt detection cycle.
Necessity	High
Priority	High
Origins	R-U-CA-01

TABLE 3.16
REQUIREMENT “R-S-FN-02”

Field	Description
Description	The system shall be able to emulate any ISA’s interrupt hook.
Necessity	High
Priority	High
Origins	R-U-CA-01

TABLE 3.17
REQUIREMENT “R-S-FN-03”

Field	Description
Description	The system shall be able to emulate any ISA’s return-from-interrupt subroutine.
Necessity	High
Priority	High
Origins	R-U-CA-01

TABLE 3.18
REQUIREMENT “R-S-FN-04”

Field	Description
Description	The system shall support multiple interrupt types.
Necessity	High
Priority	High
Origins	R-U-CA-01

TABLE 3.19
REQUIREMENT “R-S-FN-05”

Field	Description
Description	The user shall be able to enable any type of interrupt.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-01

TABLE 3.20
REQUIREMENT “R-S-FN-06”

Field	Description
Description	The user shall be able to enable all interrupts.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-01

TABLE 3.21
REQUIREMENT “R-S-FN-07”

Field	Description
Description	The user shall be able to disable any type of interrupt.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-01

TABLE 3.22
REQUIREMENT “R-S-FN-08”

Field	Description
Description	The user shall be able to disable all interrupts.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-01

TABLE 3.23
REQUIREMENT “R-S-FN-09”

Field	Description
Description	The user shall be able to clear any type of interrupt.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-01

TABLE 3.24
REQUIREMENT “R-S-FN-10”

Field	Description
Description	The user shall be able to clear all interrupts.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-01

TABLE 3.25
REQUIREMENT “R-S-FN-11”

Field	Description
Description	The user shall be able to define the ISA’s interrupt detection cycle.
Necessity	High
Priority	High
Origins	R-U-CA-01 , R-U-CA-02

TABLE 3.26
REQUIREMENT “R-S-FN-12”

Field	Description
Description	The user shall be able to define the ISA’s interrupt hook.
Necessity	High
Priority	High
Origins	R-U-CA-01, R-U-CA-02

TABLE 3.27
REQUIREMENT “R-S-FN-13”

Field	Description
Description	The user shall be able to define the ISA’s return-from-interrupt subroutine.
Necessity	High
Priority	High
Origins	R-U-CA-01, R-U-CA-02

TABLE 3.28
REQUIREMENT “R-S-FN-14”

Field	Description
Description	The user shall be able to define the ISA’s timer comparison cycle.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-03, R-U-CA-04

TABLE 3.29
REQUIREMENT “R-S-FN-15”

Field	Description
Description	The user shall be able to define the ISA’s timer hook.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-03, R-U-CA-04

TABLE 3.30
REQUIREMENT “R-S-FN-16”

Field	Description
Description	The user shall be able to define how many cycles the timer advances.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-03

TABLE 3.31
REQUIREMENT “R-S-FN-17”

Field	Description
Description	The system shall support two privilege levels of execution: user and kernel.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-05

TABLE 3.32
REQUIREMENT “R-S-FN-18”

Field	Description
Description	The system shall be able to switch between execution modes.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-05

TABLE 3.33
REQUIREMENT “R-S-FN-19”

Field	Description
Description	The system shall be able to execute system calls without interrupts.
Necessity	Low
Priority	Low
Origins	R-U-CA-06

TABLE 3.34
REQUIREMENT “R-S-FN-20”

Field	Description
Description	The user shall be able to select whether to execute with or without interrupts.
Necessity	Low
Priority	Low
Origins	R-U-CA-06

TABLE 3.35
REQUIREMENT “R-S-FN-21”

Field	Description
Description	The system shall support memory-mapped devices.
Necessity	High
Priority	Medium
Origins	R-U-CA-07

TABLE 3.36
REQUIREMENT “R-S-FN-22”

Field	Description
Description	The system’ memory-mapped devices shall provide the same capabilities as the system calls.
Necessity	Medium
Priority	Medium
Origins	R-U-CA-08

TABLE 3.37
REQUIREMENT “R-S-FN-23”

Field	Description
Description	The user shall be able to define the ISA’s device addresses.
Necessity	High
Priority	Medium
Origins	R-U-CA-09

TABLE 3.38
REQUIREMENT “R-S-NF-01”

Field	Description
Description	The system’s source code shall follow modern standards.
Necessity	High
Priority	Medium
Origins	R-U-RE-01

TABLE 3.39
REQUIREMENT “R-S-NF-02”

Field	Description
Description	The system’s source code shall be maintainable.
Necessity	High
Priority	High
Origins	R-U-RE-02

TABLE 3.40
REQUIREMENT “R-S-NF-03”

Field	Description
Description	The system shall be licensed under the LGPL-2.1 license.
Necessity	High
Priority	Low
Origins	R-U-RE-03

3.2.3. Traceability

A traceability matrix verifies if software requirements cover all user requirements and, as [Table 3.41](#) shows, all functional requirements cover the capabilities, proving the analysis was correct.

CHAPTER 4

DESIGN, IMPLEMENTATION, AND DEPLOYMENT

This chapter provides a full description of the proposed solution. It details the system's design process, by discussing the different alternatives ([Section 4.1, Study of the Solution](#)), and describes the proposed architecture ([Section 4.2, System Architecture](#)), including all components and design decisions.

4.1. Study of the Solution

In order to implement the features described in [Section 3.2, Requirements](#), the execution cycle of the simulator had to be modified, as shown in [Algorithm 4.5](#), to incorporate the interrupt, timer, and device handling subroutines.

The main change with respect to CREATOR 5's execution cycle ([Algorithm 2.1](#)) is that the interrupt detection—along with the timer and devices subroutines—, and the instruction fetch are performed at the end of the cycle instead of at the beginning of it. The execution order stays consistent: execute, check for interrupts, and fetch next instruction; but the

Algorithm 4.5: CREATOR 6's instruction execution cycle

- 1: Decode *instruction*
 - 2: Increment PC
 - 3: Execute *instruction*
 - 4: Handle timers ([Algorithm 4.7](#))
 - 5: Handle devices ([Algorithm 4.8](#))
 - 6: Handle interrupts ([Algorithm 4.6](#))
 - 7: Fetch *instruction*
-

with the current approach, the simulator can infer the state after the current instruction's execution and display it in the UI in the current cycle. This is relevant for step-by-step execution, as the UI needs to show the next instruction before the user decides to execute it. In the case of the first instruction, some extra logic is performed by the assembler in order to fetch the first instruction.

The order of the interrupt, timer, and device handling is also relevant. An interrupt can be generated as the result of executing an instruction, but timers and devices can also generate them, therefore we must check for possible interrupts *after* executing those handlers. To allow timers to interact and influence devices, the timer handler is executed before the device handler.

4.1.1. Interrupts

Simulating interrupts in a “generic” way is a somewhat difficult task. As shown in [Section 2.1](#), each ISA can define its own way of handling interrupts, but all approaches tend to converge to the same core idea: something unexpected happens and the processor either ignores it or stops execution, handles it, and resumes execution. One of the objectives of this simulator is to give the end users the ability to implement any ISA without needing to implement the whole simulator, by giving them the optimal tools for the task. The result is a “generic” interrupt model.

The approach is to divide the interrupt model into a set of common actions whose behaviour is then defined by each specific ISA. The selected interrupt handler will implement the behaviour of each action. As stated in [Section 2.1.1](#), most ISAs differentiate between different interrupt types, therefore our model must support multiple interrupt types. Some ISAs—like RISC-V—also support enabling and disabling interrupts globally, that is, all interrupt types.

The set of actions in the model is the following:

- Generating a specific type of interrupt.
- Evaluating what types of interrupts are currently pending, and returns the highest priority one.
- Clearing pending interrupts, both globally and by type.
- Handling a pending interrupt type.
- Enabling and disabling interrupts, either one specific type of interrupt, or globally.
- Evaluating if interrupts are enabled, both globally and by type.

Each ISA defines its own set of interrupt types, but in a generic model it is possible to differentiate the following¹⁶:

- *Maskable*: Generic interrupts that can be disabled.
- *NonMaskable*: Generic interrupts that can't be disabled.
- *Timer*: Interrupts generated by a timer.
- *External*: Interrupts generated by an external device.
- *Software*: Interrupts generated by software.

¹⁶Note that not all types of interrupts need to be defined in all ISAs, the approach is to allow the user to select a subset of these types.

Algorithm 4.6: CREATOR 6’s interrupt handling subroutine

```

1: if not interruptsGlobalEnabled() then
2:   | return
   end if
3: if not interruptPending() then
4:   | return
   end if
5: pending_interrupt ← getInterrupt()
6: if not interruptEnabled(pending_interrupt) then
7:   | return
   end if
8: interruptHandle(pending_interrupt)

```

- *EnvironmentCall*: A subset of *software* interrupts, in which the switch into a higher privilege mode of execution—for example, an OS—is required.

The handling subroutine, specified in [Algorithm 4.6](#), evaluates the pending interrupts and executes its handler, if interrupts are globally enabled and that type of interrupt is enabled. This algorithm only allows for one type of interrupt to be handled each time, and it’s up to the implementation of the evaluating action to apply the correct priority to each interrupt type and return the highest priority interrupt. As stated previously, instructions, timers, and devices can generate interrupts through the use of the specified action.

One of the objectives of the interrupt feature in the simulator—as stated in [Requirement R-U-CA-06](#)—is for it to be “opt-in”, that is, ensuring that the average user that doesn’t need to understand interrupts in order to use the simulator’s basic functionalities. The problem arises with the system calls, which *must* be implemented with interrupts, but are part of the basic example programs of the simulator, as they enable communication with the keyboard and display. In CREATOR 5, the system call instructions¹⁷ doesn’t make use of any type of interrupts or execution mode changes; instead, it emulates an operating system by implementing its functionality in the instruction definition¹⁸. The objective is to allow the user to implement its own interrupt handler, while also providing a “fallback” system that is transparent for the regular users.

There are two possible approaches to solving this: embedding a basic interrupt handler—written in assembly—with each program, and allowing the user to replace it; and using multiple interrupt handlers, a default one that bypasses the architecture-defined interrupt actions, and another that executes the actions, forcing the user to write their own interrupt handler. The main difference between the two approaches is that the latter allows writting the default interrupt handler in JavaScript, which in turns allows interaction with the

¹⁷ `ecall` instruction in RISC-V, `syscall` instruction in MIPS-32.

¹⁸ Currently, it uses a switch case that reads one or more specific registers and calls the desired internal functions.

Algorithm 4.7: CREATOR 6's timer handling subroutine

```

1: if not timerEnabled() then
2:   | return
   end if
3: if clk_cycles % tick_cycles = 0 then
4:   | timerAdvance()
5:   | timerHandle()
   end if

```

system through the CREATOR API (CAPI), offering the user more flexibility and ease of use. Furthermore, this approach allows switching handlers without needing to recompile, although this is perhaps only useful for debugging purposes. For the reasons mentioned, the multiple handler approach was selected.

4.1.2. Timers

As with interrupts, a generic timer model can be divided into a set of actions:

- Advancing the timer once per tick.
- Handling the different timers, typically evaluating if they reach certain threshold and generating an exception if it does.
- Enabling and disabling timers.
- Evaluating if timers are enabled.

The timer handling subroutine, specified in [Algorithm 4.7](#), would therefore advance and handle the timers once every tick—a tick being a unit of time composed of one or several clock cycles—if the timers are enabled.

4.1.3. Memory-mapped I/O

As stated in [Section 2.3](#), there are two main approaches used for I/O communication in computers: using specific ports for communication, and using memory. Ports, as they require special instructions, can be emulated in the instruction definition with CREATOR 6's plugin system¹⁹, but for ISAs that don't offer those instructions—like RISC-V—, interaction with the devices the simulator offers can only occur through the use of MMIO.

It is possible to model a “generic” device as a set of registers and a handler function. Most devices have at least a control register for the CPU to signal an action to perform, and optionally a status register for the device to broadcast its status. Instead of a single data register, and to enable more flexibility for device definition, a data range is provided, that is, a set of contiguous memory that can be used for communication and to represent several different registers. The handler is in charge of interacting with the different registers and interacting with the simulator, having *Direct Memory Access*. A device can also be enabled or disabled in the architecture definition.

¹⁹The plugin system defines custom variables and functions in order to better emulate specific architectures. These can be accessed—through CAPI—in the instruction definitions.

Algorithm 4.8: CREATOR 6's device handling subroutine

```

1: for-each device in devices
2:   if not device.enabled() then
3:     | continue
   end if
4:   device.handler()
   end for-each

```

To give the users flexibility over ISA definition, the addresses of the different devices' registers are configured in the architecture definition. This also gives the user the ability to map the registers to addresses both inside and outside main memory²⁰. For address decoding, any access to memory will first check if the address belongs to a device register, and perform the operation in the device's memory if it does.

The device handling subroutine, as shown in [Algorithm 4.8](#), executes the handler of all enabled devices once per cycle.

4.2. System Architecture

As per [Requirement R-U-RE-01](#), the system architecture of CREATOR overhauled. The application was divided into three distinct modules: a *core* module, containing all the base functionality of the simulator, and two "consumer" modules, the *Command-Line Interface* (CLI) and the *Web Application*. Both consumer modules are separate applications that act as the interface between the user and the main system's functionality (the core module)²¹.

[Fig. 4.1](#) shows the core module architecture. Its main components are the following:

- *Architecture Processor*: In charge of parsing the architecture configuration file and initializing up the rest of components according to it.
- *Register Manager*: Manages the registers and register operations.
- *Memory Management*: Manages memory access and operations.
- *Execution Engine*: Executes the loaded instructions and the rest of operations in the execution cycle, including interrupts, timers and devices.
- *Assembler*: Assembles and links the source assembly code into a set of executable instructions.
- *Sentinel*: Monitors the execution of the program, providing feedback on whether parameter passing conventions are followed.
- *CABI*: An API that defines functions to interact with the rest of the components of the system. Includes the plugin system.

4.2.1. Interrupt Manager

The Interrupt Manager's interface includes all the actions of the interrupt model mentioned in [Section 4.1.1](#), and is responsible for managing the different interrupt handlers and keep-

²⁰The size and layout of the main memory is already configured in the architecture definition.

²¹Their details are out of the scope of this thesis, but they will be briefly described in [Section 4.3](#).

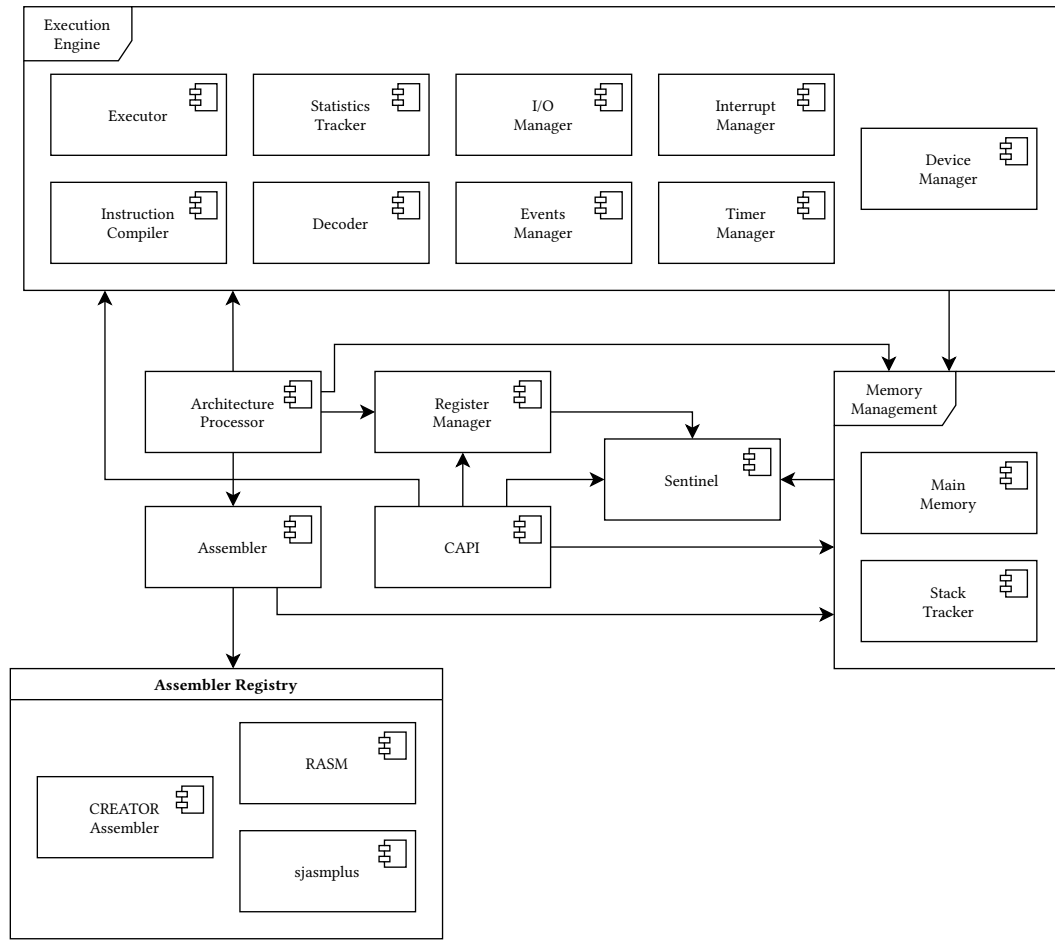


Fig. 4.1. Core module architecture

ing track of the interrupt status: if interrupts are globally enabled, which specific interrupt types are enabled, and which are pending. It also includes a reset function that clears both the internal state and any other handler or architecture-defined state, by calling the clear and enable/disable actions on the handlers—as shown in [Algorithm 4.9](#). As mentioned before, the `handle()` method implements [Algorithm 4.6](#).

When an action is performed, the interrupt manager updates its internal status and executes the corresponding action implemented by its currently selected handler. This is achieved by making use of an abstract class—as shown in [Fig. 4.2](#)—which allows maintain-

Algorithm 4.9: Interrupt Manager resetting subroutine

```

1: for-each type in interrupt_types do                                ▷ Enable all interrupts
2:   | status.enabled_interrupts[type] ← True
   end for-each
3: Synchronize interrupts (Algorithm 4.10)
4: for-each (enabled, type) in status.pending_interrupts do          ▷ Clear interrupts
5:   | status.pending_interrupts[type] ← False
   end for-each

```

ing the same interface while having completely different handler definitions. This approach also allows for easy addition of new handlers. When switching handlers, the Interrupt Manager makes use of the internal status and the model actions in order to synchronize the status—as shown in [Algorithm 4.10](#).

As mentioned before, there are two handlers implemented: the default CREATOR interrupt handler, and an architecture handler. The former handler treats all interrupt types

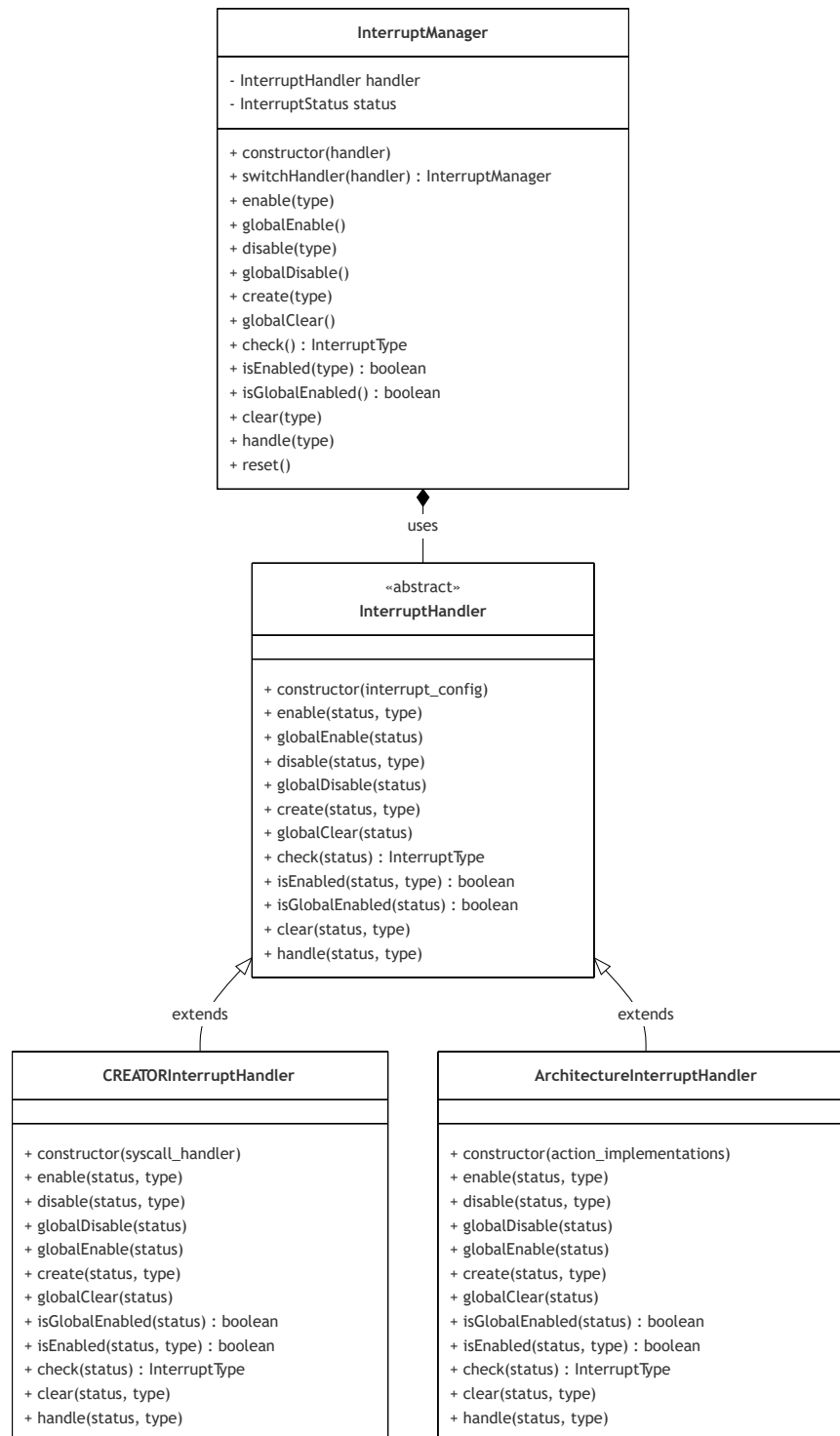


Fig. 4.2. Interrupt Manager architecture

Algorithm 4.10: Interrupt Manager syncing subroutine

```

1: if status.globalEnabled then:
2:   | handler.globalEnable()
3: else:
4:   | handler.globalDisable()
   end if
5: for-each (enabled, type) in status.enabled_interrupts do
6:   | if enabled then:
7:     | handler.enable(type)
8:   | else:
9:     | handler.disable(type)
   | end if
   end for-each

```

except *EnvironmentCall* as unexpected errors, and generates an exception that is managed by the UI. For *EnvironmentCall* interrupts, it executes the architecture-defined system call handler. The architecture handler obtains the definition of its actions from the architecture definition, and just executes them.

4.2.2. Timer Manager

The timer manager—shown in Fig. 4.3 —implements the set of actions defined in the timer model, which in turn are defined by the architecture, and the handler executes Algorithm 4.7.

4.2.3. Device Manager

The Device Manager—shown in Fig. 4.4 — is in charge of executing Algorithm 4.8, holding the information and handling all the devices defined in the architecture.

Devices all derive from the same abstract class, which implements common methods such as `isDeviceAddr()`—a method that validates if the given address belongs to the device—, a reset method—which clears the associated memory—and a clear method—which resets the control register. Each device then implements its own handler, and has its own

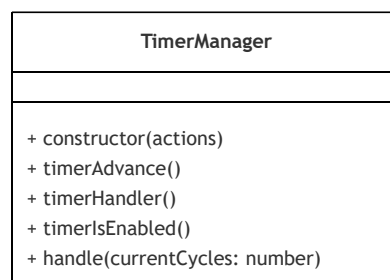


Fig. 4.3. Timer Manager

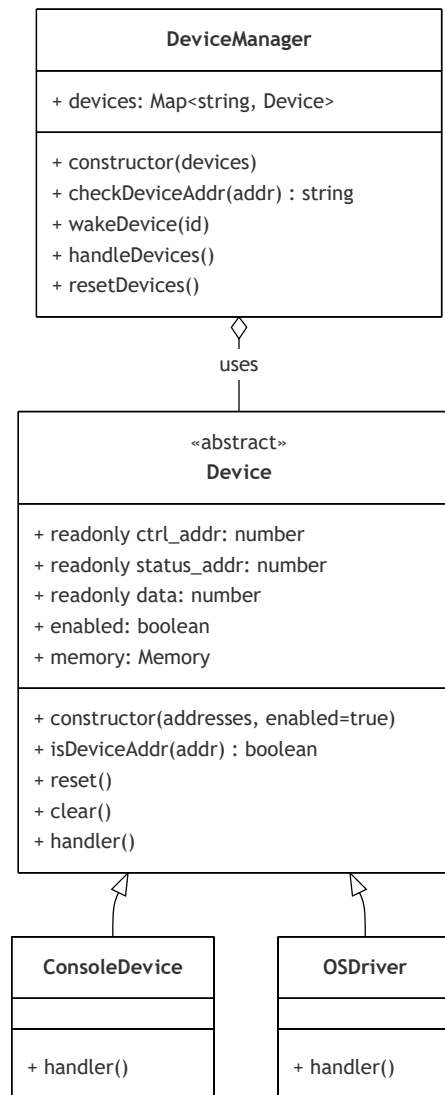


Fig. 4.4. Device Manager architecture

associated memory. This approach makes the system extensible, making it easier to add new devices.

The devices implemented are two: a console device that allows reading and writing to the UI console, and an “OS” driver to interact with the system. The console device reads the value from the control register and, depending on its value, performs the read/write operation on the specific data type, using a register in the data region to pass information, as shown in Table 4.1. The OSDriver acts similarly to the console device, and implements a couple of system operations shown in Table 4.2: *Sbrk*, which allocates memory in the heap, and *Exit*, which terminates the program.

4.2.4. CAPI . INTERRUPTS

As previously stated, CAPI—or CREATOR API—is an interface with the main system used for in the architecture definition, specially on the instructions and interrupt/timer actions. In order to allow interactions with the new features described in this thesis, a new

TABLE 4.1
CONSOLEDEVICE OPERATIONS

Control Register value	Operation	Data Register value
1	Print integer	[in] Integer value to print
2	Print float	[in] Float value to print
3	Print double	[in] Double value to print
4	Print string	[in] Address of the string to print
5	Read integer	[out] Value of the integer read
6	Read float	[out] Value of the float read
7	Read double	[out] Value of the double read
8	Read string	[out] Address of the string read
11	Print char	[in] Char value to print
12	Read char	[out] Value of the char read

TABLE 4.2
OSDRIVER OPERATIONS

Control Register value	Operation	Data Register value
9	Sbrk	[in] Size to allocate [out] Address of allocated memory
10	Exit	

module had to be added, the `CAP1.INTERRUPTS` module. This module includes the following methods:

- *setUserMode*: Sets the user privilege mode
- *setKernelMode*: Sets the kernel privilege mode.
- *create*: Creates a new interrupt of the specified type.
- *enable*: Enables the specified type of interrupts.
- *globalEnable*: Globally enables interrupts.
- *disable*: Disables the specified type of interrupts.
- *globalDisable*: Globally disables interrupts.
- *isEnabled*: Checks if interrupts of the specified type are enabled.
- *isGlobalEnabled*: Checks if interrupts are globally enabled.
- *clear*: Clears a pending interrupt of the specified type.
- *globalClear*: Clears all pending interrupts.
- *setHandler*: Sets the interrupt handler.

4.3. Implementation

CREATOR 5 was written in HTML 5 [32], CSS 2018 [33], and ECMAScript 2019 [34]²², using the Vue.js 2 web framework [35]. Instead of using ES Modules—introduced in

²²More commonly known as “JavaScript”.

ECMAScript 2015 [36]—the ECMAScript source code was divided into different logical files and concatenated before deployment. There are many shortcomings with this approach, specially for the developer experience. Having the code split in multiple unrelated files requires developers to execute a script each time they wanted to make a change in the source code, and developer tools such as linters or language servers are useless, as they don't have access to the rest of the files. Furthermore, all the variables declared in the top scope of each file were global variables, allowing accidental *shadowing*²³ and promoting accessing variables from other logical files directly instead of through curated interfaces. The Vue components were implemented inside JavaScript files instead of using Single-File Components—which require a bundler—, where the HTML definitions were defined inside JavaScript strings, requiring escaping characters, and generating an overall diminished developer experience.

The new implementation of the project is built around the Vite build tool [37]—the de facto standard for frontend tooling—, and follows modern conventions for project layout, both on the web application and the CLI. Most of CREATOR 5's code has been refactored and modified: the inclusion of ES Modules, the use of classes, Vue Single-File Components, and the addition of new tooling such as formatters and linters²⁴. The main dependencies and standards were also upgraded: Vue.js 3, CSS 2025, and ECMAScript 2025, among others. All of this has led to an improvement in reliability, maintainability, readability, scalability, and overall developer experience. Additionally, many of the files were translated into Typescript [40], which improves the robustness of the source code by adding static typing. It is also important to note the addition of the BigInt type, which allows us to implement 64-bit architectures²⁵. The CLI, on the other hand, is built on top of Deno [41], a JavaScript runtime, due to its great integration with WebAssembly [42]²⁶.

Fig. 4.5 shows the file structure of the project, in relation with the components described in Section 4.2. As CREATOR is primarily a web application, its structure mainly follows Vite's default template for web applications. Nevertheless, the source code is clearly separated into the three main modules: core, web, and CLI.

The implementation of interrupts, timers, and MMIO closely follows the designs in Section 4.2, with the inclusion of TypeScript Enums and Interfaces in order to represent the different configurations and types. For the architecture-defined implementations of timers and interrupts, the `Function` class—a safer and more efficient alternative to `eval()`—is used to create each one of the required actions, and through the use of another wrapper function, some extra variables are injected into the scope: `CAP`—a reference to `CAP`—, `status`—a view on the system's status (execution index, clock cycles, etc.)—, `registers`—a proxy for direct access to the registers—, and `InterruptType`—an Enum representing the

²³In ECMAScript and other languages, defining a variable with the same name as a previously defined variable can re-define the variable, causing undefined or unexpected behaviour.

²⁴Specifically, ESLint [38] and Prettier [39].

²⁵In ECMAScript, the `Number` type stores a 64-bit floating point (double precision) number, which only allows integers up to 53 bits.

²⁶CREATOR's default assembler [43] is written in Rust, and is compiled to WebAssembly in order to be executed in the web application. Furthermore, Deno offers a specific target for Rust WebAssembly compilation.

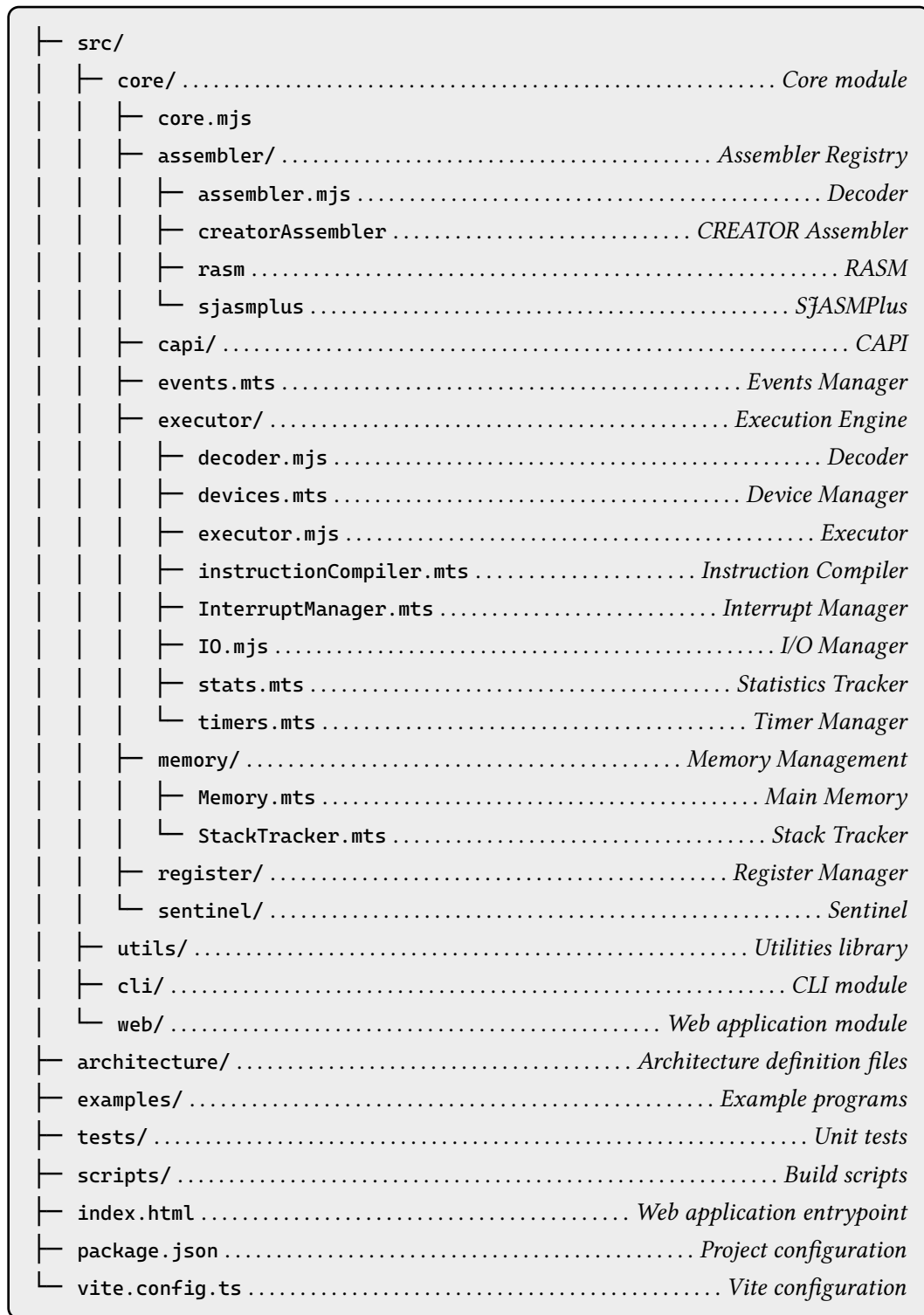


Fig. 4.5. Project file structure

different interrupt types. Finally, the `Device` class' `memory` attribute is of the same type as the main memory, as the implementation is generic enough.

CREATOR 5 included some simple regression tests through the use of the CLI, consisting of executing specific programs and comparing their outputs. The new unit tests use the Deno [41] test runner instead, improving reliability and developer experience. Some

components and classes, such as Memory also include specific unit tests to ensure their correct functionality.

4.4. Deployment

CREATOR, since its earlier versions, and being a simple web application, could be deployed on any simple web server, and it has been hosted on GitHub Pages [44], a service that can be configured to automatically deploy the latest version of a webpage whose source code is hosted in its own GitHub repository [45]. In order to reduce the size of the ECMAScript code, the Terser library [46] was used in the building pipeline. With the addition of Vite, this pipeline is simplified, and produces better results due to the fact that it also reduces the size of the HTML and CSS, while pruning unused code.

As the CLI is built on top of Deno, it is possible to generate executables for all the platforms Deno supports: Windows (x86 and ARM architectures), Linux (x86 and ARM architectures), and MacOS (ARM). These executables are built when a new version is released, and published to the GitHub repository's "Releases" page. To reduce the executable's size, only the specific core and CLI dependencies are included.

The technical specifications recommended for the final user to obtain the best experience from the application are:

- Operating System: Ubuntu 24.04 LTS (Linux distribution) / Windows 11 / MacOS Tahoe.
- Processor: Intel® Core™ i3 CPU 6300 @ 3.8GHz or higher.
- Random Access Memory (RAM): 2GB or higher.
- Storage: 100MB or free space or higher.
- Network: An Internet connection is required in order for others to access the web application, although it can be accessed locally. For deploying the web application, an Internet connection is required to download the dependencies.
- Software:
 - Chrome 145 / Safari 26.0 / Firefox 247.0 (Web application).
 - A Node.js [47] compatible package manager is required for installing the dependencies in order to deploy the web application.

CHAPTER 5

PROJECT PLAN

This chapter presents an overview of the development and logistics of the project. The planning of the project (Section 5.2, *Budget*) is detailed, analyzing its budget and overall cost (Section 5.2, *Budget*). Furthermore, it analyzes the different legislation and regulations that may apply to the project (Section 5.3, *Regulatory framework*).

5.1. Planning

This section details the project's planning, by describing the followed methodology and detailing the duration of each part.

5.1.1. Methodology

Due to the characteristics of the design, the development process was divided into four iterations:

- I. **Refactor.** This iteration's goal is to overhaul the whole project, in order to make it more maintainable and suitable for developing and feature addition. This was the longest and hardest phase, but it removed many blockers and allowed for the next phases to be quickly implemented.
- II. **Interrupts.** In this phase, the interrupt feature was implemented, including the Interrupt Manager and the associated changes in modules such as the Assembler.
- III. **Timers.** This iteration consists on implementing the Timer Manager on top of the previously implemented interrupts.
- IV. **Memory-mapped I/O.** The final iteration involved MMIO, the Device Manager, and validating the final product, seeing the three new features working together.

An iterative methodology was chosen, based on Bohem's spiral model [48], in order to ensure that, before implementing a component, all components that it depends on are correctly implemented. It also simplifies the development life cycle and makes it more

flexible, as it allows the developer to go back and modify previous elements, and encourages quick prototyping.

The life cycle development process of this model (Fig. 5.1) has four phases, which are repeated during the different iterations of the model. These phases are:

1. **Planning:** The user requirements are gathered, and the iteration's objectives are determined.
2. **Analysis:** An analysis of the user requirements is performed in order to identify potential risks, and the test cases are designed.
3. **Development and testing:** The architecture is designed, implemented, and the tests are performed.
4. **Evaluation:** The software is evaluated with the client, in order to provide feedback. In this specific case, the tutor acted as the client. This is the critical task of the life cycle, as the iteration isn't finished until the software is approved.

5.1.2. Time estimation

The time estimation of the project was designed with the use of a Gantt chart [50]. This diagram—shown in Fig. 5.2—displays all the performed tasks in each iteration of the life cycle, plus an extra documentation task in each iteration for the drafting of this thesis. A final *Report* task was added to represent the time spent finishing this and other required reports.

The project had a total duration of 13 months, with a personal dedication of 25 hours per week, totaling 1300h.

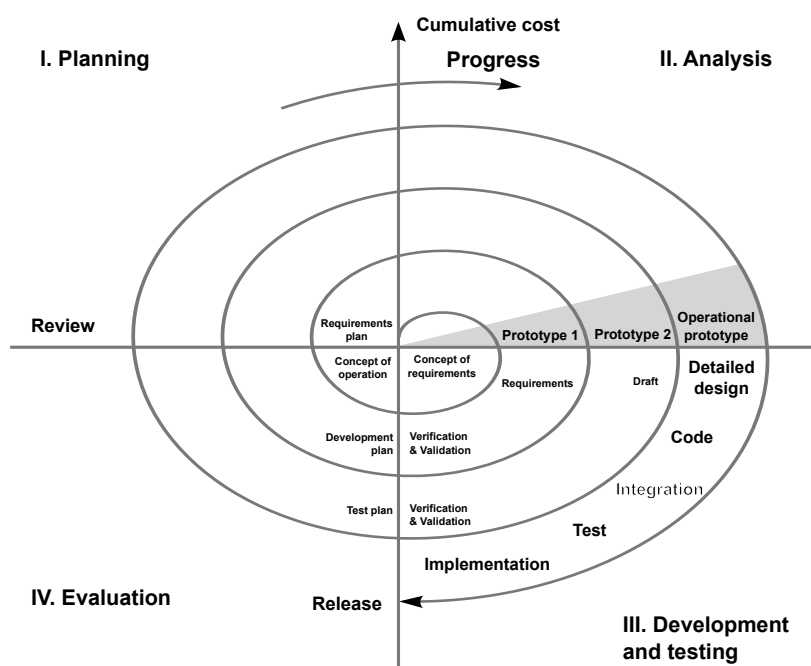


Fig. 5.1. Spiral model life cycle [49]

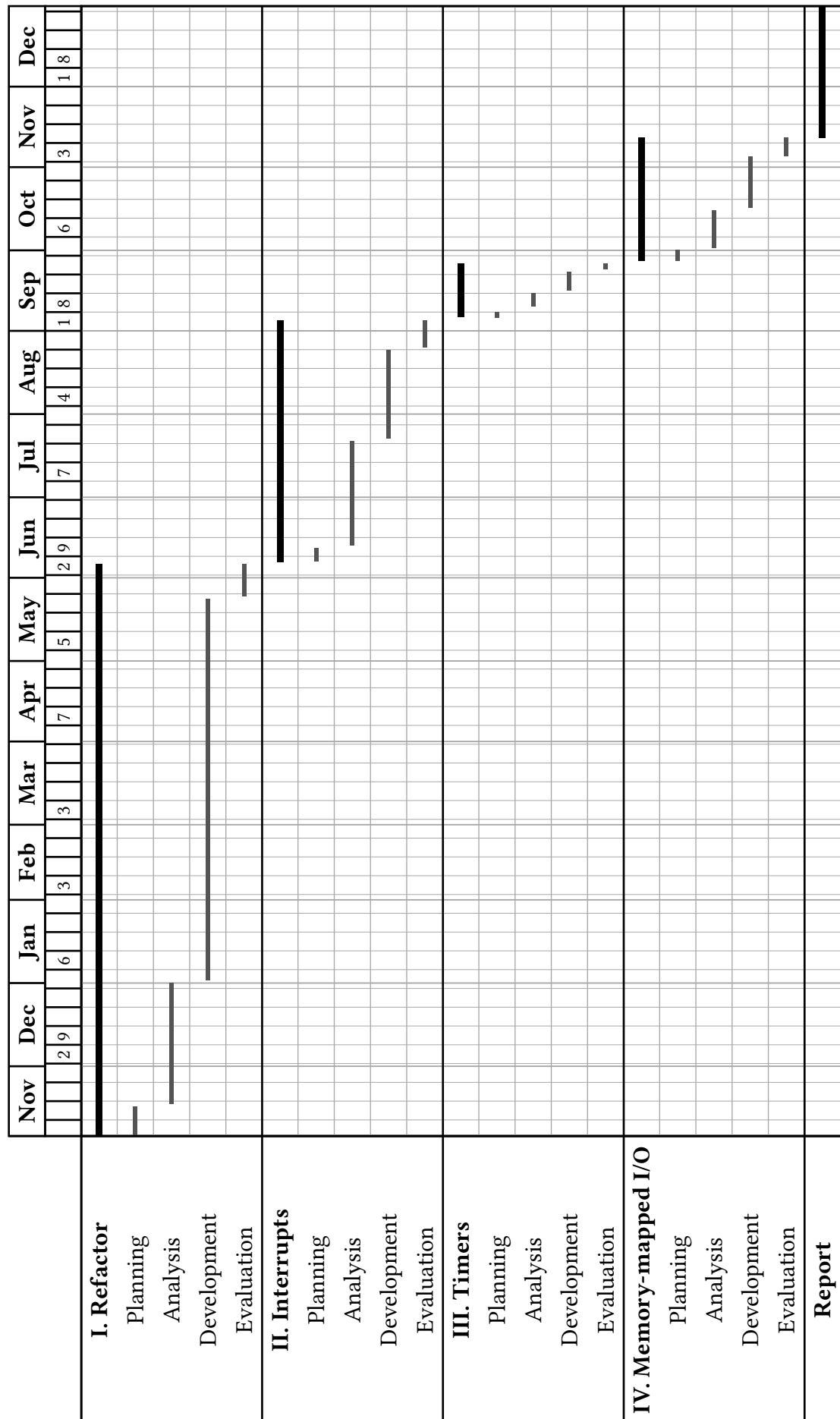


Fig. 5.2. Gantt chart for the project

5.2. Budget

This section details the project's budget, based on the time estimation and planning described in Section 5.1. Table 5.1 summarizes the main characteristics of the project, including the total budget.

TABLE 5.1

PROJECT INFORMATION

Title	<i>"Implementing Interrupts, Timers, and Memory-Mapped I/O in CREATOR"</i>
Author	Luis Daniel Casais Mezquida
Department	Departamento de Informática. Universidad Carlos III de Madrid.
Start date	November 5th, 2024
End date	December 31st, 2025
Duration	13 months
Total budget	21.060,65 €

5.2.1. Direct costs

Direct costs are those that are directly related with the development of the project. These can be divided into two groups:

- **Personnel costs:** These vary in relation to the qualifications, experience, and geographical location of each member.
- **Equipment costs:** These are all the costs associated to the tools required for developing the software, mainly hardware and software tools.

Personnel costs can be divided between four different roles²⁷:

- **Project manager:** Manages the project's schedule and provides feedback.
- **Analyst:** Analyzes the user's requirements, designs the architecture of the system, and writes documentation.
- **Programmer:** Implements the required functionalities.
- **Tester:** Designs and performs the tests for the different functionalities.

Table 5.2 shows the direct costs caused by each of the roles, and the total personnel costs.

²⁷The tutor played the role of project manager, while the student played the rest of the roles.

TABLE 5.2
PERSONNEL COSTS

Role	Hours	Cost per hour	Total
Project manager	260h	20€/h	5.200,00 €
Analyst	400h	15€/h	6.000,00 €
Programmer	700h	10€/h	7.000,00 €
Tester	200h	12€/h	2.400,00 €
TOTAL	1560h		20.600,00 €

Equipment costs refer to those caused by equipment acquisition and usage. For software, all the software tools used for this project were FOSS. The chargeable cost, C , for each item is calculated using Equation (5.1), where:

- c is the cost of the item.
- d is the time the item has been used.
- u is the percentage of the total time the item was used for the project.
- D is the item's depreciation period.

$$C = \frac{c \cdot u \cdot d}{D} \quad (5.1)$$

Table 5.3 shows the equipment costs associated to this project.

TABLE 5.3
EQUIPMENT COSTS

Item	Cost (c)	Usage (u)	Dedication (d)	Depreciation (D)	Chargeable cost (C)
Laptop	1.300,00 €	70%	13 months	84 months	140,83 €
Monitor	149,99 €	100%	13 months	96 months	20,31 €
Video cable	7,99 €	30%	13 months	36 months	0,87 €
Software	0,00 €	40%	13 months	120 months	0,00 €
TOTAL	1.457,98 €				162,01 €

5.2.2. Indirect costs

Indirect costs are those that are present during the development process, but cannot be assigned directly to any product. For the energy consumption, it is assumed that the laptop's energy usage averages 80W, and the monitor and mouse average 20W, and that they were used during the whole development process, totaling 1300h. The internet plan includes a 600mb optic fiber connection, 31.80€/month, which is shared between three people, two of them not included in the project, therefore the cost applicable to the project is a third of that.

Table 5.4 shows the indirect costs associated to this project.

TABLE 5.4
INDIRECT COSTS

Role	Hours	Cost per hour	Total
Electricity	130kWh	0.1282€/kWh	16,67 €
Internet	13 month	11.69€/month	151,97 €
Transportation	13 month	10€/month	130,00 €
TOTAL			298,64 €

5.2.3. Costs summary

Table 5.5 includes a summary of the costs of the whole project.

TABLE 5.5
COSTS SUMMARY

Personnel	20.600,00 €
Equipment	162,01 €
Indirect	298,64 €
TOTAL	21.060,65 €

5.2.4. Project offer proposal

Table 5.6 details an offer proposal for the project. This proposal includes the estimated risks, expected benefits, and taxes. After applying all these concepts, the final cost of this project, in case it is presented to a third-party client, is **35.167,07 €**.

TABLE 5.6
OFFER PROPOSAL

Concept	Increment	Partial cost	Aggregated cost
Project cost	–	–	21.060,65 €
Risk	20%	4.212,13 €	25.272,78 €
Benefits	15%	3.790,92 €	29.063,69 €
Tax	21%	6.103,38 €	35.167,07 €
TOTAL	166.98%	14.106,42 €	35.167,07 €

5.3. Regulatory framework

This section details and discusses the different regulation that may apply to the project.

5.3.1. Applicable legislation

The software can be executed locally or accessed through a public website, and it does transmit some usage information through Google Analytics [51]. As the server is treating data of citizens of the European Union, the EU's General Data Protection Regulation [52]

applies here; however, as the data collected does not concern an identified or identifiable natural person, there are no problems collecting that information. The software is also used for educational purposes, meaning there are no risks involved in the execution of the software and no other regulatory compliance is required.

5.3.2. Technical standards

The software makes use of six technical standards:

- HTML 5 [32], the latest version of the standard used to structure the web application and its content.
- CSS 2025 [53], used to control the layout, colors, and overall look of the web application's user interface.
- ECMAScript 2025 [54], the official standard for JavaScript, and Typescript [40], in which the source code for most of the application is written.
- YAML [55], which is used to store the ISA definition files and other configurable data in a format that is easy for humans to read.
- ISO/IEC 21778:2017 [56], the standard for the JSON data format, used in some of the configuration files to store and share data.

5.3.3. Licenses

CREATOR is licensed under the LGPL-2.1 license [57], in order to allow free use and modification of the source code. All of its dependencies are license through compatible licenses, such as the MIT license [58] (e.g. Vue.js).

The source code for this project is publicly available at <https://github.com/creatorsim/creator/>.

CHAPTER 6

CONCLUSIONS

This chapter presents both the conclusions of the project [Section 6.1](#), *Project conclusions*, and personal conclusions [Section 6.2](#), *Personal conclusions*. It also highlights some of the contributions that were made during this project that are not directly related to it [Section 6.3](#), *Additional contributions*, and discusses what future work could be done to the project [Section 6.4](#), *Future work*.

6.1. Project conclusions

This document has described the design, and implementation of interrupts, timers, and MMIO in the CREATOR simulator. The project presents a simple, open-source, and intuitive simulator, focused on providing the user an intuitive knowledge of interrupts, timers, and MMIO in assembly programming.

As stated in [Section 1.2](#), the main objective in this project was to add generic interrupts, timers, and MMIO to CREATOR, and this goal was reached.

All secondary objectives were also fulfilled:

- **O1:** The simulator correctly simulates both RISC-V and Z80 architectures' interrupts.
- **O2:** The simulator correctly simulates both RISC-V and Z80 architectures' timers.
- **O3:** The assembly code is able to directly interact—through memory—with the application's console.
- **O4:** The design makes use of software design patterns that make it easy to extend.

The greatest challenge was, without a doubt, the refactoring. Trying to make sense of a 90000 lines of code project, with little documentation and a *challenging* structure was a hard task. Learning to do things the right way, while doing a big refactor, while the application was still actively being developed (although, not very much) is one of those things computer scientists deal with every day, but that still hunts us in our dreams. Nevertheless,

the results speak for themselves, and the source code quality has improved greatly, which lead to faster implementation times for the rest of the features.

6.2. Personal conclusions

This project has been a great opportunity for me personally, as it has allowed not only to make use of all the knowledge I have gained throughout my degree and master, but to learn other different skills. At the beginning of the project, I had a passing knowledge of frontend development, JavaScript/Typescript development, and how to do big refactors (and not die trying). Moreover, reading and implementing technical specifications is a skill that will most surely be useful in the future, and the design process of trying to do something generic enough so it gives the required flexibility, while still trying to constrain and abstract ideas was extremely challenging, in a good way.

Lastly, being able to write a big document such as this in my beloved Typst has been a pleasure, and creating tools for other people to use, while working with other people, is one of the most fulfilling things that I can think of.

6.3. Additional contributions

This document was created and written in Typst [59]. Not only the source code of this report is publicly available at <https://github.com/ldcas-uc3m/TFM>, but the code was structured in such a way that it is easily reusable, through the use of a Typst template file that contains the configuration for the formatting of the document. This template was based on the template document provided by the Universidad Carlos III de Madrid’s library [60], and follows the master’s thesis guidelines for this university [61]. The template [62] —in collaboration with some members of the GUL-UC3M student association [63] —, along with documentation, is made publicly available in order to provide a better and easier to use bachelor and master thesis template for future engineering students.

The source code for this document also makes use of a package for automating the process of generating tables for software engineering tasks [64], such as requirements, use cases, traceability matrices, etc. This package—inspired by the author’s $\text{\LaTeX}$ version [65] — adds several features such as allowing the definition of the different items, and complete customization over how they are displayed. The source code for the package is also publicly available, in order to aid computer science students to perform these tasks in Typst.

Some other contributions²⁸ were made to the Typst package ‘rivet’ [66], a package for defining and visualizing the layout of CPU registers, which was used for the RISC-V register diagrams in Section 2.1.2.

Finally, a comprehensive documentation for the CREATOR simulator [67] was created in order to aid users of the simulator, and to showcase its new features. It includes in-depth advanced guides for defining ISAs, as well as API references and documentation for developers.

²⁸Specifically, <https://git.kb28.ch/HEL/rivet-typst/pulls/13>.

6.4. Future work

This project implemented all the features it set to, but there are several lines of work that could be explored, in order to better expand them:

- Currently, the system supports multiple interrupt types, but it only allows one of them to be active each cycle, leaving the ISA definition to decide. Perhaps the simulator could support that in a more explicit way.
- The devices' memory segments are not shown in the UI if they are not mapped to existing addresses. A custom visualizer for these devices could be implemented.
- Currently, only two devices are implemented—covering all of CREATOR's system calls —, but some other devices should be added, for example a sound board or a LED matrix, as WepSIM has.
- The new SAIL [15] execution engine makes use of preloaded kernels for its system calls and other interrupt handling. The system could also support this for the regular execution engine, as an alternative to the architecture-defined interrupt handler.
- Currently, only one timer is advanced each tick. It is possible to write the timer action definitions to support multiple timers, but this should be made more explicit in the UI, allowing more in-depth configuration.

BIBLIOGRAPHY

- [1] D. Camarmas-Alonso, F. Garcia-Carballeira, A. Calderon-Mateos, and E. del-Pozo-Puñal, “CREATOR: An Educational Integrated Development Environment for RISC-V Programming,” *IEEE Access*, vol. 12, pp. 127702–127717, 2024, doi: [10.1109/ACCESS.2024.3406935](https://doi.org/10.1109/ACCESS.2024.3406935).
- [2] RISC-V International, “History of RISC-V.” Accessed: Feb. 10, 2026. [Online]. Available: <https://riscv.org/about/history/>
- [3] “Integrated development environment for teaching and research on RISC-V processors (PDC2023-145832-I00).” Accessed: Feb. 10, 2026. [Online]. Available: <https://researchportal.uc3m.es/display/act562858>
- [4] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design (RISC-V Edition): The Hardware Software Interface*. in The Morgan Kaufmann Series in Computer Architecture and Design. Morgan Kaufmann, 2021.
- [5] P. Bulić, *Understanding Computer Organization*. in Undergraduate Topics in Computer Science. Springer International Publishing AG, 2024. doi: [10.1007/978-3-031-58075-8](https://doi.org/10.1007/978-3-031-58075-8).
- [6] J. Winans, “An Introduction to RV32I Interrupts and Traps.” Accessed: Sept. 18, 2025. [Online]. Available: <https://youtu.be/l7Jlry6PEX4>
- [7] F. Garcia-Carballeira and A. Calderon-Mateos, “Materiales para Estructura de Computadores. Grado en Ingeniería Informática, Universidad Carlos III de Madrid. Tema 4 (III): El procesador.” Accessed: Sept. 18, 2025. [Online]. Available: https://acaldero.github.io/uc3m_ec/transparencias/t4-procesador-3.pdf
- [8] “Z80 CPU User Manual,” technical report, Dec. 2016. [Online]. Available: <https://www.zilog.com/docs/z80/um0080.pdf>
- [9] “The RISC-V Instruction Set Manual Volume I: Unprivileged Architecture,” International Standard Draft, May 2025. [Online]. Available: <https://riscv.atlassian.net/wiki/spaces/HOME/pages/16154769/RISC-V+Technical+Specifications#ISA-Specifications>
- [10] “Intel® 64 and IA-32 Architectures Software Developer's Manual. Volume 3A: System Programming Guide, Part 1,” technical report, Sept. 2016. [Online]. Available: <https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-vol-3a-part-1-manual.pdf>
- [11] “The RISC-V Instruction Set Manual Volume II: Privileged Architecture,” International Standard Draft, May 2025. [Online]. Available: <https://riscv.atlassian.net/wiki/spaces/HOME/pages/16154769/RISC-V+Technical+Specifications#ISA-Specifications>
- [12] I. Logan and F. O'Hara, *The Complete Spectrum ROM Disassembly*. Melbourne House, 1983.

- [13] A. Gerlich and F. D. Cringle, “YAZE-AG - Yet Another Z80 Emulator by AG.” Accessed: Oct. 28, 2025. [Online]. Available: <https://agl.yaze-ag.de/>
- [14] J. C. Cano-Resa, F. Garcia-Carballeira, D. Camarmas-Alonso, and A. Calderon-Mateos, “Simulador web para RISC-V basado en la especificación SAIL,” in *Avances en Arquitectura y Tecnología de Computadores. Actas de las Jornadas SARTECO*, Zenodo, June 2025, pp. 367–376. doi: [10.5281/zenodo.15773218](https://doi.org/10.5281/zenodo.15773218).
- [15] REMS Project, “Sail architecture definition language.” Accessed: Sept. 24, 2025. [Online]. Available: <https://github.com/rem-s-project/sail>
- [16] RISC-V International, “RISCV Sail Model.” Accessed: Sept. 24, 2025. [Online]. Available: <https://github.com/riscv/sail-riscv>
- [17] The Regents of the University of California, “Spike RISC-V ISA Simulator.” Accessed: Oct. 03, 2025. [Online]. Available: <https://github.com/riscv-software-src/riscv-isa-sim>
- [18] “RISC-V Advanced Core Local Interruptor Specification,” International Standard Draft, Jan. 2022. [Online]. Available: <https://tools.cloudbear.ru/docs/riscv-aclint-1.0-20220110.pdf>
- [19] K. Vakil, “venus.” Accessed: Oct. 03, 2025. [Online]. Available: <https://venus.kvakil.me/>
- [20] Haoziwan, “Interactive-RISC-V-Simulator.” Accessed: Oct. 17, 2025. [Online]. Available: <https://github.com/Haoziwan/Interactive-RISC-V-Simulator>
- [21] Specy, “Asm Editor.” Accessed: Oct. 03, 2025. [Online]. Available: <https://asm-editor.specy.app/>
- [22] B. Landers, “RARS – RISC-V Assembler and Runtime Simulator.” Accessed: Oct. 27, 2025. [Online]. Available: <https://github.com/TheThirdOne/rars>
- [23] A. Calderón-Mateos, F. García-Carballeira, and J. Prieto-Cepeda, “WepSIM: Simulador modular e interactivo de un procesador elemental para facilitar una visión integrada de la microprogramación y la programación en ensamblador,” *Enseñanza y Aprendizaje de Ingeniería de Computadores*, vol. 1, no. 6, pp. 0–8, 2016, Accessed: Oct. 27, 2025. [Online]. Available: http://digibug.ugr.es/bitstream/10481/41909/1/T3_N6_Revista_EAIC_2016.pdf
- [24] A. Calderón-Mateos, F. García-Carballeira, and J. Prieto-Cepeda, “WepSIM: Web Elemental Processor SIMulator.” Accessed: Nov. 17, 2025. [Online]. Available: <https://github.com/wepsim/wepsim>
- [25] F. García-Carballeira, “WepSIM Handbook,” 2016, Accessed: Oct. 27, 2025. [Online]. Available: <https://wepsim.github.io/wepsim-manual.pdf>
- [26] T. Wilmshurst and R. Toulson, *Fast and Effective Embedded Systems Design - Applying the ARM mbed (2nd Edition)*, Second edition. Chantilly: Elsevier, 2012. doi: [10.1016/C2015-0-00101-0](https://doi.org/10.1016/C2015-0-00101-0).
- [27] X. Fan, *Real-Time Embedded Systems*. Oxford: Newnes, 2015. doi: [10.1016/C2014-0-00053-6](https://doi.org/10.1016/C2014-0-00053-6).

- [28] F. Garcia-Carballeira and A. Calderon-Mateos, “Materiales para Estructura de Computadores. Grado en Ingeniería Informática, Universidad Carlos III de Madrid. Tema 6: E/S y dispositivos periféricos.” Accessed: Dec. 25, 2025. [Online]. Available: https://acaldero.github.io/uc3m_ec/transparencias/t6-es-1.pdf
- [29] D. Black, “The ZX-Spectrum screen layout: Part I.” Accessed: Dec. 28, 2025. [Online]. Available: <https://www.overtakenbyevents.com/lets-talk-about-the-zx-spectrum-screen-layout/>
- [30] D. Black, “The ZX-Spectrum screen layout: Part II.” Accessed: Dec. 28, 2025. [Online]. Available: <https://www.overtakenbyevents.com/lets-talk-about-the-zx-spectrum-screen-layout-part-two/>
- [31] “IEEE Guide for Software Requirements Specifications,” IEEE Std. 830–1984, 1984. doi: 10.1109/IEEESTD.1984.119205.
- [32] Web Hypertext Application Technology Working Group, “HTML Standard.” Accessed: Feb. 08, 2026. [Online]. Available: <https://html.spec.whatwg.org/multipage/>
- [33] World Wide Web Consortium, “Cascading Style Sheets (CSS) Snapshot 2018.” Accessed: Feb. 08, 2026. [Online]. Available: <https://www.w3.org/TR/css-2018/>
- [34] ECMA International, “ECMAScript® 2019 Language Specification.” Accessed: Feb. 08, 2026. [Online]. Available: <https://tc39.es/ecma262/2019/>
- [35] E. You, “Vue.js 2.” Accessed: Feb. 08, 2026. [Online]. Available: <https://v2.vuejs.org/>
- [36] ECMA International, “ECMAScript® 2015 Language Specification.” Accessed: Feb. 08, 2026. [Online]. Available: <https://tc39.es/ecma262/2015/>
- [37] E. You, “Vite.” Accessed: Feb. 08, 2026. [Online]. Available: <https://vitejs.dev/>
- [38] OpenJS Foundation, “ESLint.” Accessed: Feb. 08, 2026. [Online]. Available: <https://eslint.org/>
- [39] J. Long, “Prettier.” Accessed: Feb. 09, 2026. [Online]. Available: <https://prettier.io/>
- [40] Microsoft, “TypeScript Language Specification,” technical report, 2016. [Online]. Available: <https://github.com/microsoft/TypeScript/blob/main/doc/spec-1.8.md>
- [41] Deno Land, “Deno: A modern runtime for JavaScript and TypeScript.” Accessed: Feb. 09, 2026. [Online]. Available: <https://deno.com/>
- [42] A. Rossberg, “WebAssembly Core Specification, Version 2.0,” W3C Working Draft, June 2024. [Online]. Available: <https://www.w3.org/TR/wasm-core-2/>
- [43] Á. Guerrero Espinosa, “CREATOR Assembler.” Accessed: Feb. 09, 2026. [Online]. Available: <https://github.com/creatorsim/creator-assembler>
- [44] GitHub, Inc., “GitHub Pages Documentation.” Accessed: Feb. 09, 2026. [Online]. Available: <https://docs.github.com/en/pages>
- [45] CREATOR Team, “CREATOR Repository.” Accessed: Feb. 09, 2026. [Online]. Available: <https://github.com/creatorsim/creator/>

- [46] terser.org, “Terser.” Accessed: Feb. 08, 2026. [Online]. Available: <https://terser.org/>
- [47] OpenJS Foundation, “Node.js — Run JavaScript Anywhere.” Accessed: Feb. 09, 2026. [Online]. Available: <https://nodejs.org/>
- [48] B. W. Boehm, “A spiral model of software development and enhancement,” *IEEE Computer*, vol. 21, no. 5, pp. 61–72, 1988, doi: [10.1109/2.59](https://doi.org/10.1109/2.59).
- [49] B. Boehm and W. Hansen, *Spiral Development: Experience, Principles, and Refinements*. in AD-a382 590. Carnegie-mellon univ pittsburgh pa software engineering inst, 2000. [Online]. Available: <https://books.google.es/books?id=YKmJ0AEACAAJ>
- [50] W. Clark, W. Polakov, and F. Trabold, *The Gantt Chart: A Working Tool of Management*. in Ronald manufacturing management and administration series. Ronald Press Company, 1922.
- [51] Google, “Google Analytics.” [Online]. Available: <https://analytics.google.com/>
- [52] “Regulation (EU) 2016/679 of the European Parliament and of the Council.” [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [53] World Wide Web Consortium, “Cascading Style Sheets (CSS) Snapshot 2025.” Accessed: Feb. 10, 2026. [Online]. Available: <https://www.w3.org/TR/css-2025/>
- [54] ECMA International, “ECMAScript® 2025 Language Specification.” Accessed: Feb. 08, 2026. [Online]. Available: <https://tc39.es/ecma262/2025/>
- [55] YAML Language Development Team, “YAML Ain't Markup Language (YAML) version 1.2.” Accessed: May 22, 2023. [Online]. Available: <https://yaml.org/spec/1.2.2/>
- [56] “Information technology – The JSON data interchange syntax,” International Standard ISO/IEC21778:2017, Mar. 2017.
- [57] Free Software Foundation, “GNU Lesser General Public License, version 2.1.” Accessed: Feb. 10, 2026. [Online]. Available: <https://www.gnu.org/licenses/old-licenses/lgpl-2.1.html>
- [58] Massachusetts Institute of Technology, “The MIT License.” Accessed: Feb. 10, 2026. [Online]. Available: <https://opensource.org/license/mit/>
- [59] “Typst: The new foundation for documents.” Accessed: Feb. 11, 2026. [Online]. Available: <https://typst.app/>
- [60] Biblioteca UC3M, “Master thesis template - UC3M - IEEE Style.” Accessed: Feb. 07, 2024. [Online]. Available: <https://www.overleaf.com/latex/templates/master-thesis-template-uc3m-ieee-style/fzzsvythwjyh>
- [61] Biblioteca UC3M, “Master Thesis UC3M: Writing your TFM.” Accessed: Feb. 11, 2025. [Online]. Available: <https://uc3m.libguides.com/en/MFP>
- [62] GUL-UC3M, “uc3m-thesis-ieee-typst.” Accessed: Jan. 11, 2025. [Online]. Available: <https://github.com/guluc3m/uc3m-thesis-ieee-typst>
- [63] “Grupo de Usuarios de Linux de la Universidad Carlos III de Madrid.” Accessed: Feb. 11, 2026. [Online]. Available: <https://gul.uc3m.es/>

- [64] L. D. Casais Mezquida, “srs-typst.” Accessed: Jan. 11, 2025. [Online]. Available: <https://github.com/rajayonin/srs-typst>
- [65] L. D. Casais Mezquida, “srs-latex.” Accessed: Jan. 11, 2025. [Online]. Available: <https://github.com/rajayonin/srs-latex>
- [66] L. Heredero, “RIVET – Register / Instruction Visualizer & Explainer Tool.” Accessed: Jan. 11, 2025. [Online]. Available: <https://typst.app/universe/package/rivet/>
- [67] “CREATOR Documentation.” Accessed: Jan. 11, 2025. [Online]. Available: <https://creatorsim.github.io/creator-wiki/>

APPENDIX A

DECLARATION OF USE OF GENERATIVE AI

I have used Generative AI in this work:

YES	NO
-----	----

The author of this thesis **has not** used any type of generative artificial intelligence during the development of the project or during the writing of this document.